



UNIVERSIDAD DE EXTREMADURA

Escuela Politécnica

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

Trabajo de Fin de Grado

Estudio y evaluación de técnicas básicas
de gestión de buffering y recuperación de
pérdidas en aplicaciones de streaming

Francisco Javier Ortiz Valverde

Septiembre 2025



UNIVERSIDAD DE EXTREMADURA

Escuela Politécnica

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

Trabajo de Fin de Grado

Estudio y evaluación de técnicas básicas
de gestión de buffering y recuperación de
pérdidas en aplicaciones de streaming

Autor: Francisco Javier Ortiz Valverde

Tutor: David Miguel Cortés Polo

ÍNDICE GENERAL DE CONTENIDOS

Contenido

1.	RESUMEN.....	8
2.	INTRODUCCIÓN	9
3.	OBJETIVOS	10
4.	ESTADO DEL ARTE.....	11
4.1	Análisis del retardo y métricas de transmisión	11
4.1.1	Calidad del Servicio o <i>Quality of Service (QoS)</i>	12
4.1.2	Calidad de la Experiencia o <i>Quality of Experience (QoE)</i>	12
4.2	<i>Jitter</i> o fluctuación en el retardo	13
4.3	<i>Buffering</i>	15
4.4	<i>RTP: Real-Time Transport Protocol</i>	16
4.4.1	<i>RTCP: RTP Control Protocol</i>	17
4.4.2	<i>RTSP: Real-Time Streaming Protocol</i>	17
4.5	<i>FEC: Forward Error Correction</i>	18
5.	METODOLOGÍA	20
5.1	Fase de estudio	20
5.2	Análisis del proyecto original	22
5.3	Desarrollo del proyecto final.....	22
5.4	Documentación del proyecto.....	25
5.5	Fase de pruebas	26
5.6	Estimación de esfuerzo y coste del proyecto	26
6.	IMPLEMENTACIÓN Y DESARROLLO	28
6.1	Proyecto base	28
6.2	Estructura del proyecto básico	28
6.2.1	Estados del cliente.....	30
6.2.2	Operaciones <i>RTSP</i>	31
6.2.3	Gestión del flujo <i>RTP</i>	33
6.2.4	Paquetización del contenido multimedia	33
6.3	Proyecto completo.....	36
6.3.1	Códec avanzado	36
6.3.2	Envío de información multimedia.....	39
6.3.3	Recepción de la información multimedia	39

6.4	Implementación de un de-jitter <i>buffer</i>	43
6.5	Recuperación de paquetes de vídeo mediante <i>FEC</i>	44
6.5.1	Estructura de los paquetes <i>FEC</i>	45
6.5.2	Envío de paquetes <i>FEC</i> y recuperación de vídeo.....	47
6.6	Mejoras en la interfaz.....	49
6.7	Parámetros de lanzamiento	53
6.8	Estadísticas	54
7.	RESULTADOS.....	56
7.1	Topología de la red para las pruebas.....	56
7.2	Escenario 1: Recuperación de paquetes en una red con pérdidas del 10%	59
7.3	Escenario 2: Reducción de la variación del retardo mediante un <i>de-jitter buffer</i>	68
8.	CONCLUSIONES	72
8.1	Trabajos futuros	73
9.	REFERENCIAS BIBLIOGRÁFICAS.....	74

ÍNDICE DE TABLAS

Tabla 1. Comparativa de FEC frente a retransmisión.....	19
Tabla 2. Tarifas medias por hora.....	27
Tabla 3. Estimación de horas por fase	27
Tabla 4. Cálculo de costes.....	27
Tabla 5. Resultados de las pruebas en el escenario 1.....	68

ÍNDICE DE FIGURAS

Figura 1. Funcionamiento de un de-jitter buffer	14
Figura 2. Diagrama de ciclo de vida aproximado del proyecto	20
Figura 3. Diagrama de clases del proyecto básico	29
Figura 4. Interfaz del cliente en el proyecto base.....	30
Figura 5. Diagrama de estados del cliente	30
Figura 6. Ejemplo de comunicación RTSP entre cliente y servidor	32
Figura 7. Ecuación de longitud de chunk de audio	34
Figura 8. Cabecera RTP según RFC 3550	34
Figura 9. Diagrama de clases del proyecto completo	36
Figura 10. Codificación de fichero multimedia según el códec avanzado.....	37
Figura 11. Bloque de código para procesar la cabecera del nuevo códec.....	38
Figura 12. Estructura del flujo de datos dentro del cliente	40
Figura 13. Extracto del hilo de recepción RTP del cliente extendido.....	41
Figura 14. Extracto del hilo de procesamiento del vídeo recibido.....	42
Figura 15. Funcionamiento de los de-jitter buffers.....	43
Figura 16. Diagrama del flujo FEC en RTP según RFC 5109.....	45
Figura 17. Esquema con la definición de los campos de la cabecera de primer nivel FEC	46
Figura 18. Esquema con la definición de los campos de la cabecera de segundo nivel FEC	46
Figura 19. Cálculo del número de secuencia de un paquete protegido según máscara de bits FEC.....	47
Figura 20. Estructura del flujo de datos con FEC	48
Figura 21. Interfaz renovada de estadísticas del cliente.....	50
Figura 22. Comparativa de mejoras en la interfaz de estadísticas	50
Figura 23. Gráfico de buffer de vídeo del cliente	51
Figura 24. Interfaz renovada del servidor	52
Figura 25. Menú de opciones del servidor bloqueado	52
Figura 26. Comparativa de mejoras en la interfaz del servidor	53
Figura 27. Parámetros de lanzamiento de cliente.....	53
Figura 28. Parámetros de lanzamiento de servidor	54
Figura 29. Topología de red de pruebas.....	56
Figura 30. Información de topología Mininet.....	57
Figura 31. Script de creación de la topología de pruebas	58
Figura 32. Modificación de topología para pérdidas	59
Figura 33. Comando de lanzamiento de la topología en Mininet	59
Figura 34. Enrutamiento entre subredes en Mininet.....	59
Figura 35. Secuencia de comandos para configurar el MTU.....	60
Figura 36. Comando ping de tamaño incrementado sin fragmentación.....	60
Figura 37. Estadísticas de transmisión sin FEC con 10% de pérdidas.....	61
Figura 38. Estadísticas de Wireshark en pérdidas sin FEC.....	62
Figura 39. Estadísticas de transmisión con FEC de 5 paquetes con 10% de pérdidas	63

Figura 40. Estadísticas de Wireshark con pérdidas y FEC de 5 paquetes en el flujo de redundancia	64
Figura 41. Últimos paquetes de redundancia perdidos en pérdidas con FEC de 5 paquetes.....	64
Figura 42. Paquete de vídeo perdido con FEC de 5 paquetes.....	65
Figura 43. Paquetes de vídeo perdidos recuperados con FEC de 5 paquetes	65
Figura 44. Extracto de últimos paquetes de vídeo y redundancia con FEC de 5 paquetes	66
Figura 45. Extracto de carga de paquete de redundancia 3728.....	66
Figura 46. Extracto de carga de paquete de redundancia 3738.....	66
Figura 47. Estadísticas de transmisión con FEC de 3 paquetes con 10% de pérdidas	67
Figura 48. Configuración de jitter en la topología emulada.....	68
Figura 49. Fotograma temporalmente marcado como perdido	69
Figura 50. Estado de fotograma actualizado	69
Figura 51. Extracto de consola de simulación con jitter	70
Figura 52. Paquetes fuera de orden capturados en Wireshark	71

1. RESUMEN

El presente trabajo de fin de grado aborda el estudio y la evaluación de técnicas básicas de gestión de *buffering* y recuperación de pérdidas en aplicaciones de *streaming*, con el objetivo de estudiar la calidad percibida en transmisiones multimedia en tiempo real. Con este objetivo en mente, se ha diseñado y desarrollado una aplicación experimental, compuesta por un cliente y un servidor de *streaming*, implementados sobre protocolos estándar como *RTP* y *RTSP*.

En esta plataforma se han incorporado mecanismos clave para optimizar la transmisión, entre los que destacan la implementación de un *de-jitter buffer* capaz de compensar la variabilidad en el retardo de los paquetes, y un sistema de *Forward Error Correction (FEC)*, que permite la reconstrucción de paquetes perdidos sin necesidad de retransmisiones. Todo ello fundamentado sobre la estructura de un proyecto anterior existente, que ha servido de base para el proyecto, incluyendo entre algunas de las mejoras, además de las técnicas descritas, la separación de flujos de audio y vídeo, la incorporación de un códec avanzado, y la renovación de la interfaz de usuario y estadísticas de transmisión.

Las pruebas se han llevado a cabo mediante escenarios de red emulados, evaluando métricas como la latencia, *jitter* y pérdida de paquetes con el fin de comprobar la eficiencia de los mecanismos implementados. Los resultados obtenidos pretenden demostrar que, el uso de *buffers* adaptativos y *FEC*, contribuye significativamente a una reproducción fluida y estable, incluso en condiciones de red adversas.

Este trabajo no solo recopila las bases teóricas, necesarias para comprender las técnicas de optimización de transmisiones multimedia, sino que también proporciona la perspectiva práctica, permitiendo implementar y optimizar dichas funcionalidades en futuros sistemas de transmisión en tiempo real, donde la calidad de experiencia del usuario es un factor crítico.

2. INTRODUCCIÓN

Es sabido que el consumo de contenido multimedia en tiempo real ha crecido en los últimos años de forma exponencial, debido a la popularización de plataformas de *streaming*, videollamadas y transmisiones en directo. Este crecimiento ha supuesto para los desarrolladores, un desafío a la hora de cumplir con las expectativas de calidad del cliente y aspectos como la latencia, *jitter* o pérdidas de paquetes pueden afectar significativamente a la calidad percibida por el usuario, especialmente en aplicaciones que requieren de una mayor interactividad y fluidez.

Para hacer frente a estos problemas, múltiples técnicas de gestión de las transmisiones multimedia han surgido en los últimos años, destacando entre ellas el uso de *buffers* adaptativos, para compensar la fluctuación en el retardo, así como la aplicación de mecanismos de corrección de errores, que permiten recuperar la información perdida o errónea en entornos donde la retransmisión no es viable, ya sea por limitaciones de tiempo o por el tipo de conexión.

En esta memoria se presenta el proyecto desarrollado en este trabajo de fin de grado, en el que, mediante la implementación de un software real, se demuestra tanto la funcionalidad, como la efectividad de las distintas técnicas empleadas para la mejora de calidad en transmisiones multimedia, brindando el proyecto un punto de vista práctico en el que ha sido aplicado el marco teórico ya existente.

La estructura general de esta memoria comienza estableciendo, en primer lugar, los objetivos del trabajo, definiendo las limitaciones y metas por las que se ha regido el desarrollo del proyecto. A continuación, se aclaran y definen los conceptos básicos en el contexto de la transmisión multimedia, explicando las nociones fundamentales relacionadas al trabajo, necesarias para comprender su desarrollo y funcionalidad.

Tras el estado del arte, en el capítulo cinco, se describe de forma general la metodología en la que se ha trabajado en proyecto, definiendo las distintas fases estructuradas en las que el trabajo ha sido construido. Abarcando desde la investigación inicial, pasando por el desarrollo del software, en el que se basa el trabajo, y finalizando con las fases de documentación y pruebas.

Seguido de la metodología, se encuentra el cuerpo del trabajo, en el que se describe el proyecto inicial del que parte el proyecto, además de cómo ha sido extendido a las necesidades y objetivos del trabajo, describiendo la estructura del software desarrollado, y estableciendo cómo se han implementado las bases teóricas de forma práctica en el proyecto.

Finalmente, esta memoria concluye con la descripción y resultados de una serie de pruebas realizadas en entornos emulados, tratando de mostrar la efectividad de las técnicas implementadas en un contexto de ejecución real. Mostrando el proceso de análisis y los valores obtenidos en el proceso, son finalmente extraídas las conclusiones que se han podido derivar de estos.

3. OBJETIVOS

El objetivo principal de este proyecto es el de analizar, implementar y evaluar técnicas básicas de *buffering*, además de la recuperación de pérdidas, en el contexto de las aplicaciones de *streaming*, con el fin de mejorar la calidad de las transmisiones multimedia y la experiencia final del usuario.

Con este objetivo general en mente, se plantean los siguientes objetivos específicos:

- **Definir el estado del arte** de los mecanismos y protocolos empleados en la transmisión multimedia.
- **Diseñar y desarrollar un software como prueba de concepto**, que permita simular, en un entorno controlado, las distintas técnicas discutidas en este trabajo.
- **Implementar un *de-jitter buffer*** que permita estabilizar la entrega de paquetes, reduciendo los efectos del *jitter* en la reproducción del flujo multimedia.
- **Desarrollar un protocolo de corrección de errores hacia adelante**, que permita la reconstrucción de paquetes, sin necesidad de una retransmisión por parte del servidor.
- **Evaluar la eficacia de las soluciones implementadas** mediante pruebas, en entornos emulados, mediante métricas objetivas en distintas condiciones de red.
- **Sentar una base para posibles mejoras y futuras investigaciones**, permitiendo aportar un enfoque práctico que sirva para perfeccionar y ampliar las técnicas implementadas.

4. ESTADO DEL ARTE

A lo largo de este apartado, se tratarán los conceptos y cuestiones de transmisión multimedia asociados a la temática del trabajo, con el fin de facilitar la comprensión de este y dar un mejor contexto al proyecto.

4.1 Análisis del retardo y métricas de transmisión

El retardo, en el contexto de telecomunicaciones, es entendido como el tiempo transcurrido desde que se transmite un dato de un punto a otro de una red determinada. Este retardo entre dos puntos puede deberse a múltiples factores, por lo que, conocer los distintos tipos de retardo, nos permite diagnosticar y diseñar soluciones para cada configuración posible.

Se considera el **retardo de transmisión**, al tiempo en el que se demora un nodo emisor en disponer un conjunto de datos en el medio de transmisión [1]. Esto podría ser, por ejemplo, el tiempo que tarda un router en convertir los datos en señales eléctricas en el cable de cobre, o el tiempo que tarda en transformar la información en ondas y enviarlas al aire, para el caso de las conexiones inalámbricas. Este retardo depende directamente de la velocidad de transmisión del equipo, la cual se expresa en *bits/segundo* o *bps*, dicha velocidad debe adaptarse a la capacidad del medio de transmisión y a la del nodo receptor [2]. Esto quiere decir que, aunque el equipo emisor pueda tener una mayor capacidad de envío, debe adaptar su velocidad a la máxima permitida por el medio de transmisión y capacidad del destino para recibir los datos.

Por otra parte, el **retardo de propagación** es el tiempo empleado durante una transmisión para que los datos viajen a través del medio por el que se están desplazando [2]. El medio puede ser cualquiera que permita el envío de datos, un par de cobre, fibra óptica e incluso el aire, como es en el caso de las transmisiones inalámbricas. El retardo de propagación se ve directamente afectado por la distancia física entre los dos nodos y por el tipo de medio por el que se transmite, por lo que no se puede reducir de forma notable en la mayoría de los escenarios.

Existe además el **retardo de procesamiento**, que representa el tiempo que tarda el nodo en tratar la información, esto puede ser para analizarla, modificarla, reenviarla, etc. El retardo de procesamiento normalmente es introducido en la red por los equipos encargados de dirigir la información, encontrando en el caso de las redes de datos a los routers y los conmutadores en las redes de telefonía [2], además de mantener una relación inversa con la capacidad de procesamiento del nodo, reduciendo el retardo a mayor sea la potencia del equipo que procesa.

Finalmente, el **retardo de cola** aparece en los escenarios en los que es necesario esperar a la disponibilidad del medio o del nodo, antes de poder transmitir los datos [1]. El retardo de cola será menor en escenarios donde no se comparte el medio de salida, o donde el nodo receptor no impone limitaciones en la tasa de recepción. [2]. Un ejemplo práctico del retardo de cola, podría ser la espera de un paquete en un router antes de ser transmitido, debido a otros paquetes que deban ser transmitidos antes al cable.

4.1.1 Calidad del Servicio o *Quality of Service (QoS)*

La Calidad de Servicio, o *Quality of Service (QoS)*, hace referencia al conjunto de métricas técnicas y objetivas empleadas para determinar el desempeño promedio y fiabilidad de una red. En el contexto de multimedia, estas métricas permiten garantizar una calidad aceptable para servicios como voz o vídeo en tiempo real, *streaming* o videoconferencias [3].

A continuación, se listan algunas de las principales métricas empleadas para evaluar *QoS*, junto a los síntomas típicos que causan en el ámbito de la multimedia:

- **Pérdida de paquetes:** Causa artefactos visuales e interrupciones [4, 5].
- **Jitter o variación de retardo:** Afecta a la sincronización y fluidez del vídeo [4, 5].
- **Latencia o *delay*:** Degrada la interactividad, especialmente notable en entornos de *streaming* en vivo o videollamadas [4, 5].
- **Ancho de banda y *Throughput*:** Determinan la capacidad para enviar datos [5].

Algunos protocolos, como *RTCP*, hacen uso de estas métricas para mantener un informe en tiempo real sobre la calidad de la transmisión, con el fin de diagnosticar y subsanar posibles cambios en la red que puedan interferir con el correcto funcionamiento del servicio [6]. Actualmente, existen estándares de *QoS* como *ITU-T G.1010* o *ITU-T G.1070*, que establecen los requisitos mínimos u óptimos para los distintos servicios multimedia [4].

En la práctica, las medidas objetivas que ofrece *QoS* se relacionan con medidas subjetivas de *Quality of Experience (QoE)*, como el *Mean Opinion Score (MOS)*, con el objetivo de realizar mejoras desde una perspectiva orientada a la experiencia final del usuario.

4.1.2 Calidad de la Experiencia o *Quality of Experience (QoE)*

La Calidad de Experiencia, o *Quality of Experience (QoE)*, se entiende como la medida subjetiva del grado de satisfacción del usuario respecto a un servicio o aplicación. Esta medida, incluye la totalidad de efectos del sistema extremo a extremo y puede verse alterada por el contexto y expectativas del usuario, lo que provoca que la medida pueda diferir entre distintos usuarios [7].

Esta forma de entender el software, orientada a la experiencia del usuario, queda definida por el *Qualinet White Paper* como el grado de satisfacción o insatisfacción del usuario respecto a una aplicación o servicio. Aclarando que es el resultado del cumplimiento de las expectativas del usuario, con respecto a la utilidad y/o disfrute de la aplicación o servicio, teniendo en cuenta la personalidad y estado actual del usuario [8].

“Quality of Experience (QoE) is the degree of delight or annoyance of the user of an application or service. It results from the fulfillment of his or her expectations with respect to the utility and / or enjoyment of the application or service in the light of the user’s personality and current state.” [8]

- Qualinet White Paper on Definitions of QoE (2012/2013)

A diferencia de *QoS*, que describe el comportamiento técnico de la red a través de medidas objetivas como la latencia, *jitter*, pérdida o ancho de banda, *QoE* interpreta la percepción humana del servicio y cómo estas variaciones técnicas impactan en la experiencia final [9]. Por ejemplo, dos servicios con valores de *QoS* similares, pueden ser percibidos de forma distinta por los usuarios según factores contextuales, expectativas o el propio contenido [8].

La evaluación de *QoE* puede abordarse de forma subjetiva empleando escalas sobre los usuarios como el *Mean Opinion Score* o *MOS*, siguiendo la recomendación *ITU-T P.800* [10, 11], u optar por una vía más objetiva, mediante modelos computacionales, que estiman la calidad percibida a partir de parámetros de red y de aplicación, como en el *E-model (ITU-T G.107)* [11].

Aplicado a los servicios multimedia en tiempo real, la *QoE* resulta crítica, pues cambios en el retardo, pérdidas o fluctuaciones, pueden impactar drásticamente la experiencia del usuario.

4.2 Jitter o fluctuación en el retardo

En el ámbito de las telecomunicaciones, se denomina *jitter* a la variación del tiempo de transmisión de los paquetes, produciendo una tasa inconsistente de llegada de datos. El *jitter* puede resultar especialmente problemático para ciertas aplicaciones, ya que los paquetes pueden llegar demasiado pronto o tarde para su uso, además de la posibilidad de que no lleguen en orden, debido a variaciones en el medio de transmisión o peticiones de reenvío de datos.

El remedio contra el *jitter* más extendido en las aplicaciones multimedia es la implementación de un *buffer* de datos que, a coste de aumentar el retardo, estabiliza la tasa de recepción a un intervalo constante de los paquetes. Al *buffer* empleado para este propósito se le denomina *playout delay buffer* o *de-jitter buffer* [1]. En la Figura 1 se ilustra, de forma simplificada, cómo el *de-jitter buffer* corrige las variaciones en el tiempo de la transmisión, reteniendo los paquetes temporalmente y permitiendo extraerlos uno a uno a un ritmo y orden predecible.

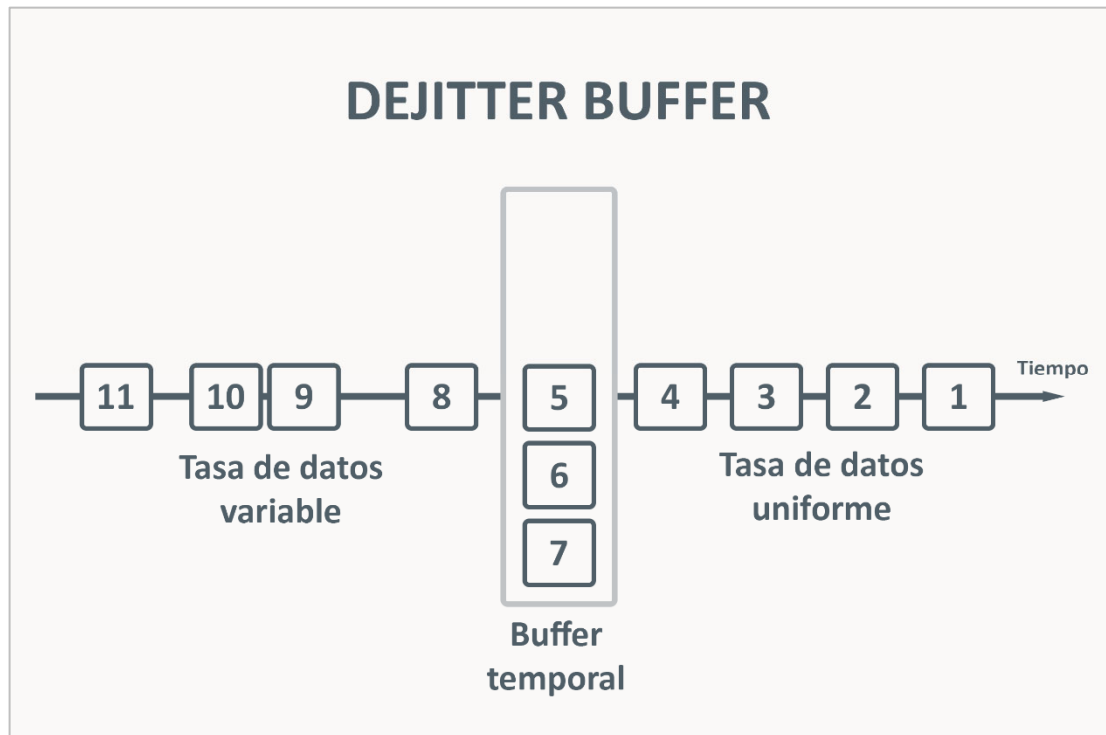


Figura 1. Funcionamiento de un de-jitter buffer

Aunque el uso de un *de-jitter buffer* puede mitigar las irregularidades en el tiempo de llegada de los paquetes, es importante definir el rango del *buffer*. Si el retardo de un paquete es demasiado grande, como para ser recibido fuera del rango del buffer, será descartado, dado que llega demasiado tarde como para reproducirlo manteniendo la uniformidad, generando interrupciones en la transmisión. En cambio, si se trata de eliminar este caso con un rango de *buffer* demasiado amplio, se introduce un retardo adicional en la reproducción que, en ciertos escenarios podría resultar inadmisible [12].

En ciertos casos, el *de-jitter buffer* puede ser adaptativo con un retardo máximo establecido [12], haciendo que la reproducción se mantenga lo más uniforme posible, teniendo en cuenta los cambios en las condiciones de la conexión.

4.3 Buffering

El *buffering* es una técnica empleada en las transmisiones multimedia, la cual consiste en la carga de datos con anterioridad en memoria, con el objetivo de garantizar una reproducción fluida y sin interrupciones. El uso de esta técnica resulta indispensable para compensar las fluctuaciones de la red, actuando como un mediador entre la transmisión y la reproducción [13].

El funcionamiento del *buffer* es sencillo, se trata de un espacio de memoria reservado, en el que se almacenan temporalmente los paquetes que contienen los datos multimedia, previo a que sean procesados por el reproductor. Permitiendo liberar los paquetes al reproductor al ritmo deseado y dar algo de margen para solucionar problemas de paquetes erróneos, perdidos o tardíos.

Actualmente existen múltiples implementaciones de *buffering*, adecuadas para distintos escenarios y necesidades:

1. **Buffering previo (Pre-buffering):** Se carga una cantidad fija de datos antes del inicio de la reproducción. Técnica bastante común en plataformas de vídeo a demanda para evitar cortes más adelante, al coste de un tiempo extra en el inicio de la reproducción [14].
2. **Buffering adaptativo:** Se ajusta dinámicamente el tamaño del *buffer* según las condiciones actuales de la red en tiempo real. Esta implementación, algo más compleja, es ampliamente usada en servicios de *streaming* en directo [15].

El *buffering* juega un papel esencial en la calidad percibida por los usuarios de los servicios de *streaming* por múltiples razones, como la estabilidad en la reproducción y la compensación en la fluctuación de la velocidad de transmisión [13]. Además de todo ello, permite la transmisión de formatos como *4K* o *HDR*, los cuales requieren de un mayor ancho de banda y que, de no ser por las técnicas de *buffering*, no sería posible su reproducción fluida [15, 16].

Sin embargo, estas ventajas no vienen sin un coste, pues el uso del *buffering* puede presentar ciertas limitaciones y problemas que, de no implementar adecuadamente el *buffer* puede suponer una disminución de la calidad en la experiencia del usuario:

1. **Buffer underflow:** Sucede cuando el *buffer* se vacía por completo a causa de que los datos no llegan con suficiente rapidez, provocando cortes en la reproducción [13, 15, 16].
2. **Buffer overflow:** Ocurre cuando el *buffer* se llena por completo y se intenta almacenar más información de la que puede contener, potencialmente provocando pérdida de información, lo que puede derivar en cortes y artefactos visuales o auditivos en la transmisión [13].
3. **Latencia inicial:** El uso de un *buffer* excesivo al inicio puede aumentar el tiempo de espera antes de que comience la reproducción, perdiendo la sensación de inmediatez del usuario [15].

4. **Consumo de recursos:** El almacenamiento y gestión de datos en memoria temporal requiere de recursos del dispositivo, lo que podría suponer un problema de rendimiento al emplear *buffers* grandes en equipos con recursos limitados [14, 15, 16].

Con estas limitaciones en mente, es fundamental adaptar los parámetros del *buffer* según las características de la transmisión y la red. Adicionalmente, muchos proveedores han optado por emplear técnicas complementarias al *buffer*, como el uso de códecs que permitan una mejor compresión, reduciendo el tamaño de los datos sin comprometer la calidad, o tecnologías de *streaming* de tasa de bits adaptable como *DASH* (*Dynamic Adaptive Streaming over HTTP*) o *ABR* (*Adaptive Bitrate Streaming*), ajustando la calidad del vídeo según los recursos disponibles de la conexión en tiempo real [14, 15, 16].

4.4 RTP: Real-Time Transport Protocol

RTP o *Real-Time Transport Protocol* es un estándar de red desarrollado por el *IETF* (*Internet Engineering Task Force*), publicado en 1996 (*RFC 1889*) y con el objetivo de servir de transporte de flujos multimedia en tiempo real como transmisiones en vivo, videoconferencias o *VoIP*, sobre redes *IP*. Este protocolo del nivel de aplicación (o capa siete del modelo *OSI*), fue diseñado con la intención de poder gestionar la sincronización, además de facilitar la compensación del *jitter*, la detección de pérdida de paquetes y resolver la entrega de paquetes fuera de orden, situaciones especialmente comunes en las transmisiones *UDP* en las que opera *RTP* [17, 18].

Aunque originalmente se publicó como un protocolo multidifusión, *RTP* se emplea frecuentemente en aplicaciones unidifusión, entornos de videoconferencia y sistemas pulsa y habla, dada su priorización a la baja latencia sobre la fiabilidad absoluta, permitiendo algunas pérdidas con el fin de mantener la fluidez en la reproducción.

El paquete *RTP*, mediante una cabecera sencilla con la información esencial, permite funciones como la detección de pérdidas, reorganización de paquetes fuera de orden, marcas temporales para la sincronización de flujos, múltiples fuentes de sincronización y contenido y ampliación de funcionalidad mediante protocolos complementarios, como *RTCP* para un mejor control del flujo y calidad, *SRTP* para la implementación de protecciones en entornos sensibles, o *RTSP* para el control de transmisiones en tiempo real [18, 19].

La versión inicial de *RTP* fue definida en el *RFC 1889* en 1996, aunque en 2003 se publicó la revisión *RFC 3550* que sustituyó a la original. Aunque esta actualización no modificó la estructura base del protocolo ni del formato de los paquetes, estableció de forma más concreta el uso de este y mejoras en *RTCP* para gestionar la conexión de gran cantidad de participantes simultáneos [20].

4.4.1 RTCP: RTP Control Protocol

RTCP o *RTP Control Protocol* es, como su nombre indica, un protocolo complementario a *RTP*, diseñado exclusivamente para el transporte de información de control y monitoreo del contenido multimedia, sin transmitir este. Definido bajo el estándar *RFC 1889* y actualizado en el *RFC 3550*, la función principal de *RTCP* es la recopilación de estadísticas de la transmisión, como número de bytes y paquetes enviados, perdidos y *jitter*, entre otros, lo que permite evaluar la calidad de una sesión *RTP* [20, 21, 22, 23].

Mediante esta retroalimentación, las aplicaciones pueden realizar cambios en tiempo real que permitan adaptarse a las condiciones de la red durante la transmisión. Por ejemplo, el uso de resolución adaptativa según las condiciones de la red en la que se transmite [21, 22].

Algunas de las funciones que brinda *RTCP*, además de proporcionar información estadística de la transmisión, permite la coordinación y sincronización de distintas fuentes de multimedia procedentes del emisor, además de la identificación de este. Todas estas funciones no solo permiten las funciones mencionadas de adaptación dinámica, optimizándola frente a condiciones adversas de la red, o la sincronización de flujos, sino que la gestión del ancho de banda que proporciona ayuda a evitar la sobrecarga de red en sesiones multidifusión, aún con una gran cantidad de participantes [20, 21, 22].

4.4.2 RTSP: Real-Time Streaming Protocol

RTSP, o *Real Time Streaming Protocol*, es un protocolo de control de red desarrollado por el *IETF (Internet Engineering Task Force)* perteneciente a la capa de aplicación (o nivel siete según el modelo *OSI*), que habilita el control de servidores multimedia mediante comandos sencillos propios de un reproductor de vídeo, como *PLAY* y *PAUSE*, entre otros [19, 24].

Es importante recalcar que *RTSP* es un protocolo no orientado a conexión y no se encarga de la transmisión de los datos multimedia, sino que únicamente gestiona el flujo de estos manteniendo una sesión asociada a un identificador. *RTSP* trabaja en conjunto de otros protocolos encargados del transporte como *RTP* para coordinar la transmisión multimedia. *RTSP* ha sido ampliamente adoptado en el ámbito de la videovigilancia y cámaras *IP*, permitiendo controlar la reproducción en tiempo real [19, 24].

La primera especificación de *RTSP* se publicó en 1998 como *RFC 2326* donde se definió un protocolo similar a *HTTP* en sintaxis, diseñado para el envío de solicitudes bidireccionales entre cliente y servidor [19, 25]. Años más tarde, en 2016, la especificación sería actualizada en la *RFC 7826*, dejando obsoleto el protocolo original y estableciendo las bases para *RTSP 2.0*, con mejoras en la gestión de sesiones y una mayor claridad en cuanto a cómo debería comportarse el protocolo [19, 24, 26].

Algunas de las operaciones más usadas en *RTSP* son:

- **DESCRIBE** para solicitar una descripción del contenido apuntado por una dirección *RTSP*.
- **SETUP** para establecer los parámetros de cómo será transportado el flujo de datos. Esta especificación típicamente incluye un puerto para recibir datos por *RTP* y otro para control con *RTCP*.
- **PLAY** indica al servidor que debe comenzar el envío de datos a través de los flujos especificados en *SETUP*.
- **PAUSE** detiene temporalmente uno o todos los flujos.
- **TEARDOWN** por otra parte detiene por completo la entrega de datos, liberando los recursos asociados.

Además de estas operaciones existen otras tantas que permiten gestionar los flujos de datos, aunque estas son las más típicas [19, 25, 26].

4.5 FEC: Forward Error Correction

Forward Error Correction (FEC), o corrección de errores hacia adelante, es una técnica de control de errores en sistemas de transmisión de datos, que permite que el receptor detecte y corrija una cantidad limitada de errores, sin necesidad de retransmisión, mediante la inclusión de información redundante en el mensaje original por parte del emisor [27, 28, 29].

La forma en la que *FEC* funciona, es mediante la inserción de bits de paridad al flujo de datos original, añadiendo redundancia controlada en los datos. Estos bits de paridad se calculan mediante distintos algoritmos *FEC* como códigos *Reed-Solomon*, *Hamming*, *Turbo*, *LDPC* o incluso implementaciones más sencillas como la paridad *XOR*, empleada en el *RFC 5109* [27, 28, 29, 30]. Una vez generados dichos códigos, estos son insertados en el flujo de datos, ya sea en el mensaje o como un paquete por separado, según el algoritmo. Al otro lado de la transmisión, el cliente puede aplicar el mismo algoritmo, empleando los datos redundantes para reconstruir la información errónea o faltante [28, 30].

El uso de corrección de errores hacia adelante supone una herramienta fundamental en la transmisión multimedia moderna, especialmente en escenarios donde la velocidad, fiabilidad y baja latencia son críticas o la retransmisión no es posible. Como se muestra en la siguiente *Tabla 1*, la corrección de errores hacia adelante representa una gran alternativa a la retransmisión en entornos donde, reenviar la información faltante sería imposible o produciría un retardo inaceptable, abarcando mayoritariamente los escenarios de transmisión en tiempo real [29, 31].

Características	<i>Forward Error Correction</i>	Retransmisión (<i>ARQ</i>)
Retransmisión	No requerida	Sí siempre que hay error
Latencia	Muy baja	Mayor (tiempo de retransmisión)
Uso de la red	Mayor debido a la redundancia	Menor, dado que solo se usa si se requiere retransmisión
Tipo de red en el que resalta	Enlaces de baja fiabilidad o de alta latencia	Enlaces más fiables con baja latencia
Aplicabilidad	Streaming en tiempo real y transmisiones unidireccionales	Transmisiones bidireccionales sin necesidad de baja latencia

Tabla 1. Comparativa de FEC frente a retransmisión

Además de las transmisiones en tiempo real, *FEC* es usado hoy en día en aplicaciones como comunicaciones por satélite, llamadas *VoIP*, redes *4G* y *5G* e incluso fuera del ámbito de las transmisiones, como es el caso de las memorias *NAND flash*, *CDs* y *DVDs*, donde se usa la corrección de errores para subsanar problemas en memoria o daños físicos en la unidad [27, 29, 31].

5. METODOLOGÍA

Para el desarrollo del proyecto, se ha seguido una metodología de desarrollo dividida en una serie de fases. Dichas fases, al ser bien definidas, permiten un desarrollo progresivo y organizado de la aplicación.

Aunque no se ha llevado un registro exacto del ciclo de vida del proyecto, es posible representar, de manera aproximada, cómo se han distribuido los tiempos para las distintas fases del trabajo. Representación ilustrada en la siguiente *Figura 2*.

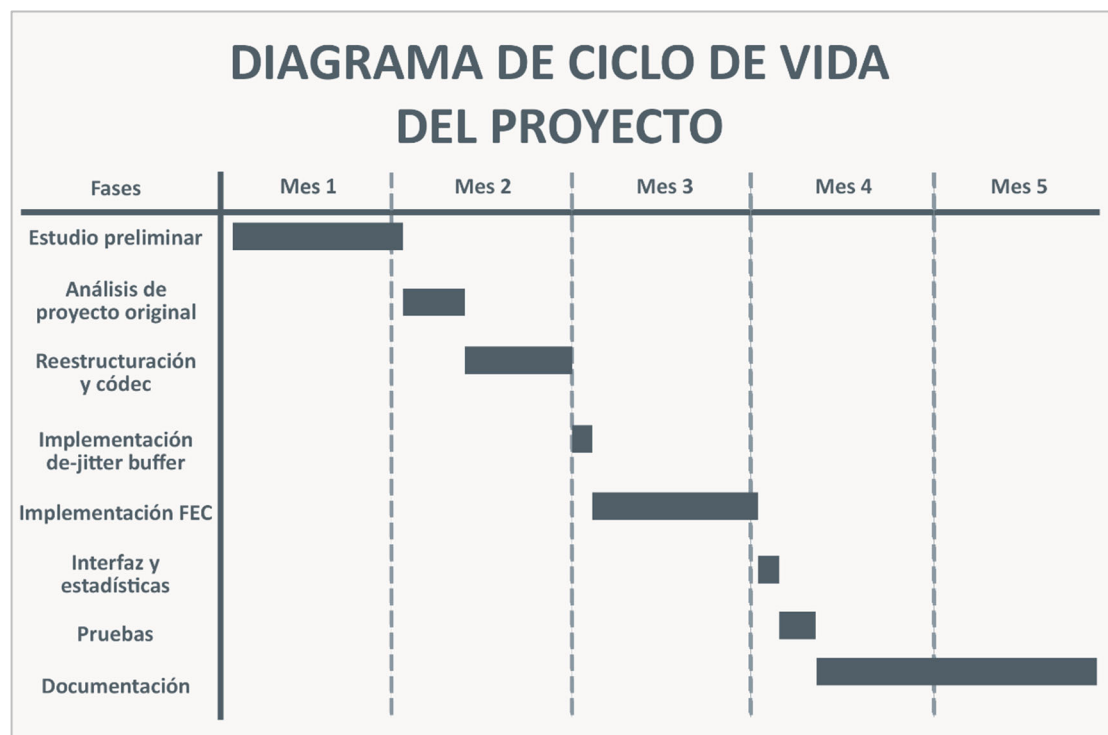


Figura 2. Diagrama de ciclo de vida aproximado del proyecto

A lo largo de los siguientes apartados, se detallan las distintas fases por las que se ha compuesto el ciclo de desarrollo del trabajo y del programa en el que apoya.

5.1 Fase de estudio

En la fase inicial de estudio, se tiene como objetivo el análisis de los objetivos del proyecto, tratando de definir los requisitos necesarios para lograrlos y las técnicas y tecnologías involucradas.

En esta etapa se estudiaron las distintas tecnologías existentes en el ámbito de la transmisión multimedia, teniendo el objetivo del proyecto en mente, permitió seleccionar las técnicas a implementar y determinar la magnitud del trabajo. Esta investigación se centró en los siguientes aspectos:

- **Transmisiones *RTP* y *RTSP***, dado que ya se conocía su uso en proyecto base sobre el que se iba a construir el trabajo, era fundamental conocer el protocolo empleando para poder realizar las expansiones pertinentes.
- **Métricas *QoS*** como forma de extraer resultados de las técnicas implementadas, además de ser necesario para la implementación de dichas métricas en el programa desarrollado.
- ***Buffering* y corrección de variación de retardo**, siendo uno de los puntos fuertes del proyecto y fundamental para la posterior implementación de un *de-jitter buffer*.
- **Técnicas de corrección de errores**, permitiendo el estudio de la técnica de *Forward Error Correction* o *FEC*, la implementación de la recuperación de datos sin reenvío por parte del cliente mediante redundancia controlada, resultando crucial en el entorno de transmisión en tiempo real en el que se enmarca el proyecto original.
- **Simulación de redes y entornos controlados**, pues todas las pruebas necesitaban ser desarrolladas en un entorno con factores controlados, necesario para poder comprobar la efectividad de las diversas técnicas en un entorno real, adaptado a las necesidades de la prueba.

5.2 Análisis del proyecto original

El análisis del proyecto original se trató de una etapa crucial para el posterior desarrollo del trabajo, pues era imprescindible familiarizarse con el proyecto original al completo. Comprender la estructura y protocolo empleado en el código original supuso también, entender la codificación específica de la información multimedia que se iba a emplear pues, por cuestiones demostrativas, no se empleó un códec oficial, sino un códec simplificado, optimizado para el uso requerido por el proyecto.

Además de esto, fue necesario también comprender cómo la modificación propuesta por el tutor, que habilitaba al proyecto del manejo de audio y vídeo simultáneo, implicaba también la comprensión de un nuevo códec personalizado, en conjunto con un cambio de estructura en el proyecto original.

El estudio se realizó en su mayoría sobre el propio código del proyecto original y a través de la documentación proporcionada por el tutor, que mostraba cómo se establecían las comunicaciones entre el cliente y servidor a grandes rasgos, además de definir la disposición de los datos codificados, para ambos códecs personalizados. Esto permite el uso del códec avanzado para las futuras expansiones.

Esta fase sentó las bases de la implementación y ayudó a diseñar la estructura que seguiría el proyecto final.

5.3 Desarrollo del proyecto final

El desarrollo del proyecto final experimentó una serie de fases durante su maduración. A continuación, se detallan las distintas etapas por las que pasó el desarrollo del código del cliente y servidor, realizados a lo largo de este trabajo.

Antes de comenzar el desarrollo, se establecieron los objetivos de esta fase, priorizando al inicio la implementación del códec avanzado proporcionado por el tutor, de forma que el proyecto permitiera la transmisión de audio y vídeo. El código base era limitado en funcionalidad y difícilmente podía soportar la expansión de las funcionalidades de una forma que cumpliera los requisitos. Resultaba evidente que la implementación del nuevo códec conllevaba la necesidad de una reestructuración del proyecto original.

Tras la reestructuración inicial, se concluyó que la implementación de un *de-jitter buffer* sería un buen inicio, dado que se trataba de una implementación sencilla de realizar y proporcionaba rápidamente resultados respecto a los objetivos marcados. Más adelante habría que afrontar la corrección de errores hacia adelante, que debía subdividirse en varios aspectos:

- **Construcción de la estructura de paquetes *FEC*:** Era necesario crear una clase que representara el paquete de redundancia, encargada de calcular la secuencia de bytes correspondiente a la información.

- **Gestión de la información de redundancia:** Tanto para la construcción del paquete de redundancia en el lado del servidor, como para la recuperación de la información en el lado del cliente, era crítico establecer las bases de cómo se gestionarían los paquetes de vídeo, pues era necesario almacenarlos hasta que fuera necesario su uso por parte del protocolo. Esto conlleva, además, gestionar una correcta liberación de los recursos para evitar sobrecargar la memoria.
- **Definición del funcionamiento del protocolo:** Para poder hacer uso de la redundancia, era necesario establecer las directrices de cómo debía construirse, y transportarse la información de redundancia por la red, además de entre los hilos del cliente, esto con el objetivo de no perder información y realizar una implementación limpia y ordenada.

Las primeras horas del desarrollo se dedicaron enteramente a la implementación del códec avanzado dado por el tutor. Esto era fundamental para la gestión simultánea de vídeo y audio, antes limitada a un único flujo por transmisión.

En esta fase, se realizaron múltiples modificaciones a la estructura general del proyecto, pues había que gestionar dos flujos de manera simultánea. Inicialmente, se diseñó de forma que un único hilo de envío se encargara de enviar todo el audio y vídeo periódicamente, pero esto rápidamente se descartó pues, tratar el vídeo y audio en el mismo hilo no solo suponía un aumento en el retardo de procesamiento notable, sino también arrastraba la limitación de que ambos flujos debían tener la misma frecuencia, algo que no todos los medios tienen.

Finalmente, se estableció una estructura de varios hilos, donde cada uno lee y envía la información del fichero a la frecuencia deseada y con números de secuencia independientes. Esta misma lógica fue también trasladada al cliente, donde se diseñó un sistema de tres hilos, donde un hilo principal recibe la información de red y la redirecciona a los hilos de procesamiento, según su contenido. Esto resultaría muy útil más adelante para la implementación de *buffering* y la corrección de errores hacia adelante.

La reestructuración con múltiples hilos derivó inmediatamente al desarrollo del *de-jitter buffer* pues, con una estructura de varios hilos, era necesario definir una estructura intermedia entre ellos, de modo que el hilo de recepción pudiera almacenar los paquetes para su posterior uso por los hilos de procesamiento. Las estructuras empleadas para esta función fueron colas de prioridad, debido a su naturaleza ordenada, resultan ideales para reordenar los paquetes mediante número de secuencia, aunque lleguen fuera de orden y, dado que el hilo de procesamiento únicamente lee los paquetes de manera periódica según los necesita, el sistema de compensación de *jitter* quedaba implementado fácilmente, gracias a la sólida estructura desarrollada, basada en hilos independientes.

En este punto, comienza el grueso de la implementación, en el que se implementa el protocolo de *Forward Error Correction*, empleando como referencia el *RFC 5109* [30], de forma que se comenzó por crear una nueva clase, como abstracción de lo que un paquete de redundancia debería contener. Inicialmente se sopesó la idea de compartir herencia con la ya existente clase para los paquetes *RTP*, pero por cuestiones de funcionalidad fue rápidamente descartado, los paquetes de redundancia debían ser empaquetados en *RTP*.

La implementación de esta clase conllevó una más detallada investigación del funcionamiento del protocolo, especialmente el correcto uso y cálculo de los distintos valores alojados en los campos de la cabecera *FEC*. Además de implementar la creación de paquetes, se documentaron internamente todos los campos y se desarrolló el proceso de cálculo de redundancias, así como la función de recuperación dados los paquetes del grupo protegido. Pruebas unitarias fueron desarrolladas con el fin de comprobar el correcto funcionamiento de la clase pues, de existir errores en el cálculo de la información de los paquetes, podría acarrear problemas difíciles de diagnosticar más adelante. Los test unitarios se desarrollaron en JUnit y sirvieron para asegurar la funcionalidad de la clase.

Una vez completado el desarrollo de la funcionalidad de los paquetes de redundancia, era preciso enviarlos de forma periódica y gestionar los paquetes de vídeo para su uso. Por parte del servidor, necesitaba almacenar temporalmente los paquetes que han sido enviados para poder calcular la información de redundancia, mientras que, en el cliente, resultaba imprescindible gestionar los paquetes recibidos para que no fueran descartados antes de que el paquete de redundancia pudiera ser procesado. Para resolver estos problemas, se creó una lista en el servidor que almacenara tantos paquetes como requiriera para completar un grupo *FEC*. En esta lista se inserta cada paquete enviado hasta que se cumple el cupo del grupo, momento en el que se genera el paquete de recuperación con la información de los paquetes almacenados y se envía, liberando entonces la lista para el siguiente grupo.

En cuanto al cliente, siguiendo el esquema de concurrencia establecido, se creó un nuevo hilo de procesamiento, esta vez encargado de gestionar los paquetes de redundancia. Para que este hilo funcionase correctamente, era necesario mantener los paquetes de vídeo que habían llegado más allá del tiempo de su reproducción, permitiendo ser empleados para recuperar los datos de un paquete futuro perteneciente al mismo grupo, por lo que se creó una lista que contuviera temporalmente los paquetes de vídeo hasta que el hilo de procesamiento *FEC* los reclamase, o hasta que no fueran relevantes.

En cuanto al hilo de procesamiento de los paquetes *FEC*, la funcionalidad es similar a los otros dos hilos pues, cuando el hilo de recepción obtiene un paquete que contenga información de redundancia, este es enviado al *buffer* que contiene los paquetes *FEC*, es entonces cuando despierta el hilo de procesamiento correspondiente y verifica si los paquetes del grupo se encuentran en la lista de protegidos, recuperando el paquete perdido al *buffer* de vídeo si fuera necesario y liberando la memoria del grupo en la lista de paquetes protegidos.

Con todo esto, la técnica de *Forward Error Correction* quedó implementada, no sin altibajos en el proceso, problemas como la sincronización de flujos o gestión de los recursos de los hilos supusieron retos durante la implementación de la corrección de errores hacia adelante.

Dadas las nuevas capacidades tanto del cliente como el servidor, estaba claro que las estadísticas originalmente mostradas quedaban totalmente obsoletas. La ventana de estadísticas del cliente era rudimentaria, no mostraba la información de los flujos por separado y, por supuesto, no contaba con las nuevas funcionalidades implementadas. Además de la interfaz del cliente, la ventana del servidor era escasa y poco clara, no podía ser usada como fuente de información fiable ni tenía un uso como tal.

Por todo esto se mejoró el sistema de estadísticas del cliente, no solo separando las estadísticas por flujos y añadiendo nuevas métricas que acompañasen a las funcionalidades, sino que también se renovó por completo la interfaz de la ventana, mejorando la claridad, estética y bajando la frecuencia de actualización para no colapsar los hilos de procesamiento o dibujado de la interfaz.

Además, el servidor recibió también una mejora de interfaz significativa en este punto, añadiendo un panel de opciones y estadísticas más claras que permitieran un mejor seguimiento del comportamiento de este.

En este punto, se consideró que el programa del proyecto cumplía con los objetivos del trabajo. En consecuencia, se declaró concluida la fase de desarrollo del software.

5.4 Documentación del proyecto

Ya desarrollado el software, se inició la documentación del estado del arte. Esta fase se caracteriza por un fuerte énfasis en la investigación y recopilación de información de fuentes fiables, que permitiera dar un contexto al trabajo y explicara, de forma más detallada, el origen, definición y funcionalidad de los distintos conceptos que forman la base del trabajo, tratando de plasmar los conocimientos necesarios para comprender tanto los objetivos, como las decisiones de implementación en el trabajo.

Además del estado del arte, se documentó todo el proceso de implementación del proyecto desarrollado, haciendo énfasis en los conceptos prácticos y soluciones implementadas para las distintas técnicas y protocolos. Esta fase, que tiene su origen en la etapa de implementación, tenía como objetivo mostrar el trabajo realizado a lo largo del proyecto, además de intentar desglosar las especificaciones concretas de cómo se ha logrado desarrollar e implementar los conceptos teóricos de forma práctica, como un software real.

5.5 Fase de pruebas

La fase de pruebas, como su nombre indica, consistió en la emulación de situaciones reales en las que poder poner en práctica las técnicas desarrolladas en el proyecto.

Antes de realizar las pruebas, era fundamental poder crear un entorno controlado, en el que no intervinieran factores externos al realizar las mediciones necesarias. Por esta razón, se empleó la herramienta de emulación de topologías de red *Mininet* [32], un software de código abierto creado inicialmente por *Bob Lantz* como prototipo y cuya versión inicial fue lanzada junto a *Brandon Heller*.

Mediante una interfaz de consola, *Mininet* habilita la creación y emulación de topologías de red completas con todo tipo de componentes, incluyendo la emulación de *hosts*, *OpenFlow* y *SDN*, entre otras funciones [32]. Empleando esta herramienta, se creó una topología de pruebas en la que se simulaban situaciones de red adversas, permitiendo mostrar la efectividad de la corrección de errores y la atenuación del *jitter*.

Junto a *Mininet*, también se hizo uso de la herramienta complementaria *MiniNAM* [33, 34], de código abierto y que permite, mediante una interfaz gráfica, mostrar visualmente la topología emulada por *Mininet* en tiempo real, habilitando una mejor gestión y monitorización del entorno de simulación.

MiniNAM fue desarrollada por el *Mobile & Internet Systems Laboratory (MISL)* en marzo de 2017, siendo el *MISL* el departamento de ciencias de la computación dentro de la *Universidad de Cork* en Irlanda.

Además de estas herramientas, se hizo uso del ya conocido analizador de red de código abierto, *Wireshark* [35], desarrollado por el equipo del mismo nombre, *The Wireshark Team*. Usando *Wireshark* se estudió, de forma externa a la aplicación desarrollada, el flujo de paquetes y su comportamiento de punto a punto.

En esta fase, se realizaron las pruebas en el entorno emulado y se analizaron los resultados extraídos, de los que se dedujeron conclusiones. Todo ello se documentó correspondientemente en los apartados finales de este trabajo, mostrando desde cómo se realizó la preparación del entorno de pruebas, pasando por el análisis de las distintas métricas y finalizando con las conclusiones y comparativas, extraídas de la realización de las pruebas.

5.6 Estimación de esfuerzo y coste del proyecto

Dado que se trata de un proyecto de software, a continuación, se desglosa la estimación de esfuerzo y el coste asociado, suponiendo el desarrollo completo del trabajo por parte de una empresa de ingeniería.

Basado en el diagrama del ciclo de vida del proyecto estimado de la *Figura 2*, se estima que una empresa de ingeniería habría requerido de un equipo reducido, formado por los siguientes componentes mostrados en la *Tabla 2*, junto a sus tarifas por hora aproximadas basadas en la media europea.

Componente	Tarifa media por hora
Ingeniero Sénior	65€/h
Ingeniero Júnior	40€/h
Especialista en Redes	50€/h
Diseñador UI	45€/h
Gestor de proyecto	55€/h

Tabla 2. Tarifas medias por hora

Establecidas las tarifas, se estimó que, dado que el proyecto ha sido realizado por una única persona, un equipo reducido debería completarlo en mucho menos tiempo. Con esta idea en mente, se realizaron las siguientes estimaciones de tiempo de la *Tabla 3*.

Fase del proyecto	Sénior (h)	Júnior (h)	Redes (h)	UI (h)	Gestión (h)	Total horas
Estudio preliminar	25	15	0	0	5	45
Análisis del proyecto original	10	10	0	0	5	25
Reestructuración y códec	40	30	0	5	5	80
Implementación <i>de-jitter</i> buffer	15	10	0	0	3	28
Implementación <i>FEC</i>	50	35	10	0	5	100
Interfaz y estadísticas	20	20	0	20	3	63
Pruebas	10	15	20	0	5	50
Documentación	50	65	10	10	5	140
Totales estimados	220	200	40	35	36	531 h

Tabla 3. Estimación de horas por fase

Con esta distribución de horas en mente, se puede desglosar los costes totales del proyecto según las tarifas establecidas. En la siguiente *Tabla 4* se muestra el cálculo aproximado de costes de desarrollo.

Rol	Tarifa €/h	Horas	Coste (€)
Ingeniero Sénior	65 €	220	14.300 €
Ingeniero Júnior	40 €	200	8.000 €
QA / Redes	50 €	40	2.000 €
Diseñador UI/UX	45 €	35	1.575 €
Gestión de proyecto	55 €	36	1.980 €
Total estimado			27.855 €

Tabla 4. Cálculo de costes

6. IMPLEMENTACIÓN Y DESARROLLO

A lo largo de este apartado se desglosa cómo se ha realizado la implementación según los objetivos del proyecto, desarrollando las técnicas de corrección de errores y *buffering* sobre la estructura del proyecto inicial.

6.1 Proyecto base

Esta implementación tiene su base en un proyecto anterior, preparado por el tutor *David Miguel Cortés Polo*, como apoyo para impartir la parte práctica de la asignatura de *Redes de Comunicaciones Móviles y Multimedia*. Dicho proyecto tenía como objetivo estudiar acerca de las transmisiones multimedia, empleando *RTSP* o *Real Time Streaming Protocol*.

Este proyecto base, una vez completado según los requisitos de las prácticas de laboratorio, se compone de un cliente y un servidor *RTP/RTSP* multimedia simples, desarrollados en Java, creados con el objetivo de analizar y entender el flujo de las transmisiones multimedia sobre *RTP*, todo ello bajo distintos escenarios de red emulados. De forma similar, la implementación realizada sobre este proyecto base, a lo largo de este trabajo nos permite simular y experimentar con las distintas técnicas en escenarios ficticios, así como recoger la información necesaria para extraer conclusiones.

6.2 Estructura del proyecto básico

El proyecto básico, compuesto por un cliente y un servidor, se realizan las comunicaciones entre ambos mediante dos *sockets*: uno dedicado al flujo de vídeo o audio a través de *RTP*, y otro encargado de gestionar las comunicaciones *RTSP* para el control de la transmisión. La conexión *RTP* se encapsula, a su vez, en un paquete *UDP*, permitiendo una transmisión más rápida, ya que no es necesario confirmar la llegada de cada paquete multimedia, mientras que la información del flujo *RTSP* se envía mediante *TCP* para evitar la pérdida de comandos. Se puede ver el esquema de clases del proyecto representado en la siguiente *Figura 3*.

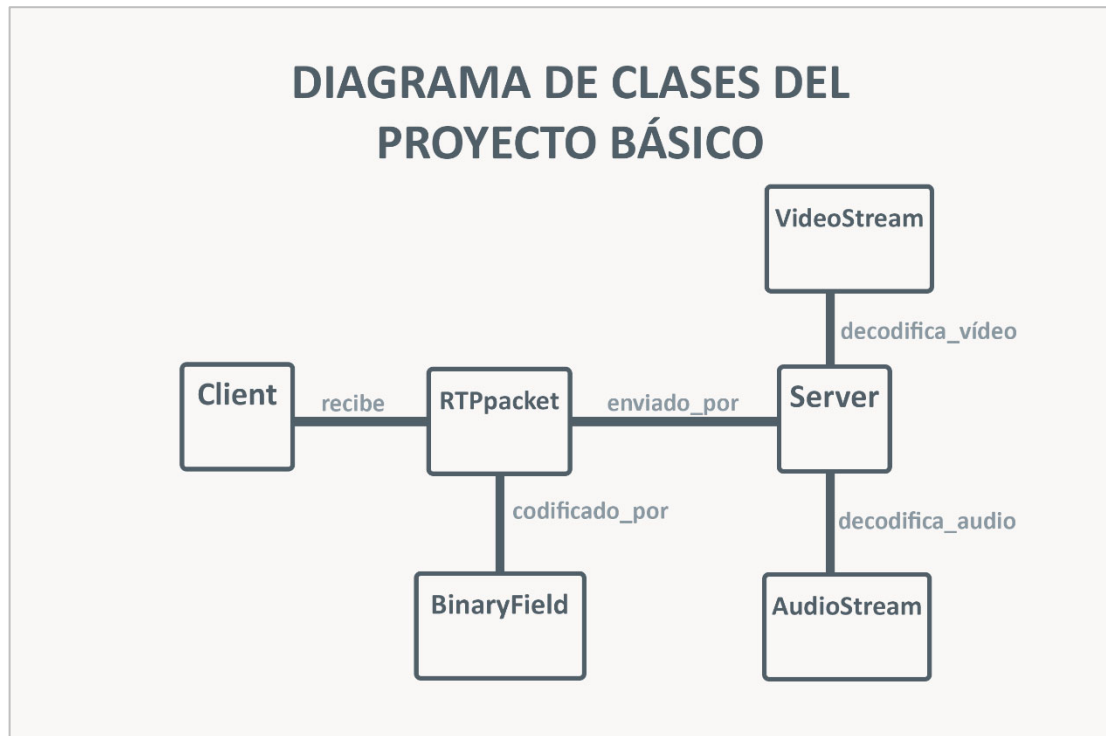


Figura 3. Diagrama de clases del proyecto básico

Inicialmente, el proyecto base permitía únicamente la reproducción de vídeo o audio, proponiéndose opcionalmente, la implementación de un códec didáctico que permitía la reproducción de vídeo y audio de forma simultánea. Este trabajo parte del proyecto base sin expansiones, aunque dicho códec fue implementado, siguiendo las directrices propuestas en la documentación de la asignatura, para soportar simultáneamente audio y vídeo.

El cliente se trata de un reproductor de vídeo básico, dispone de una interfaz, mostrada en la *Figura 4*, que cuenta con un espacio para el vídeo, además de botones que permiten controlar las operaciones básicas para manejar la transmisión (*SETUP*, *PLAY*, *PAUSE* y *TEARDOWN*). Añadido a esto, y con el objetivo de poder estudiar lo que sucede en la red, dispone de una ventana auxiliar donde se muestran, de forma rudimentaria, estadísticas relevantes sobre la transmisión en tiempo real.

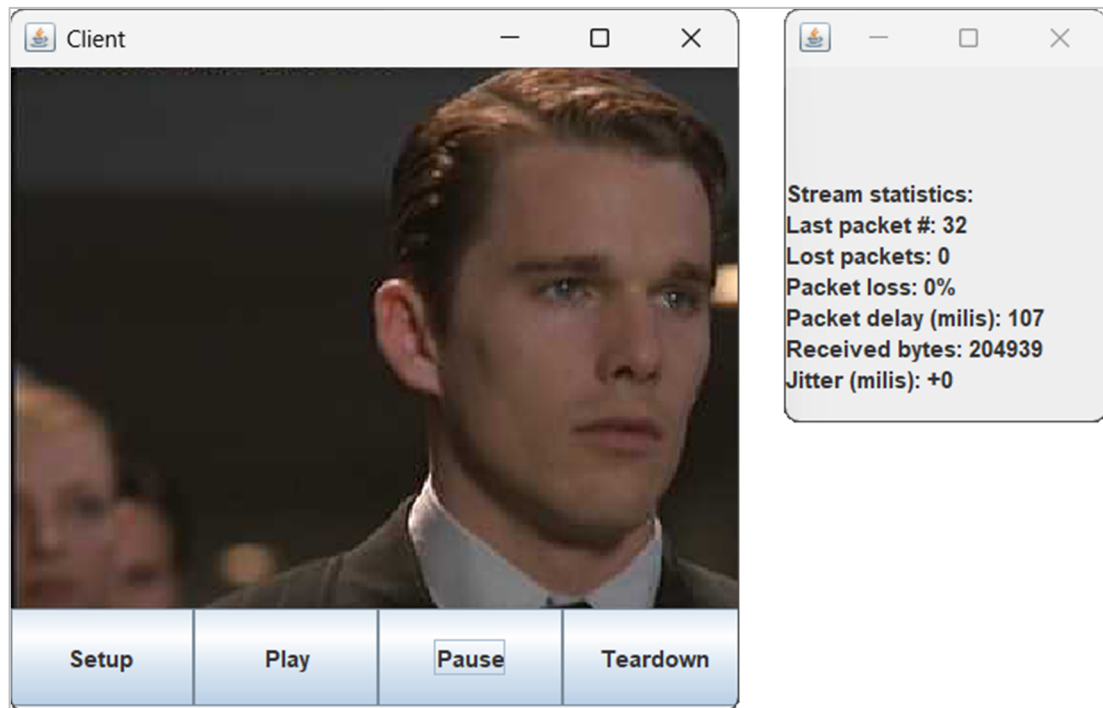


Figura 4. Interfaz del cliente en el proyecto base

Bajo la interfaz, el cliente básico funciona según la máquina de estados mostrada en la *Figura 5*, que basa sus transiciones entre estados en las distintas operaciones disponibles, correspondientes a solicitudes *RTSP* del cliente hacia el servidor.



Figura 5. Diagrama de estados del cliente

6.2.1 Estados del cliente

- **INIT:**

Estado inicial del cliente, previo a establecer la conexión *RTP*, encargada de transmitir el flujo de datos multimedia. A este estado únicamente se vuelve al finalizar el cliente.

- **READY:**

Estado en el que el cliente tiene establecida la conexión *RTP*, aunque sin reproducir vídeo ni audio. Durante este estado el servidor no envía información de vídeo ni audio al cliente, y este no reproduce contenido.

- **PLAYING:**

Representa el estado en el que el cliente se encuentra reproduciendo audio, vídeo, o ambos. En este estado, el servidor envía continuamente información multimedia para que el cliente pueda consumirla al ritmo apropiado.

6.2.2 Operaciones *RTSP*

- **SETUP:**

Permite establecer la conexión *RTP* inicial para el posterior intercambio de información. Ejecutando la acción *SETUP* en el estado *INIT*, el cliente establece el *socket* para recibir la información *RTP*, enviando una petición *SETUP* al servidor con la información requerida de la conexión. Al recibir la respuesta del servidor, esta se procesa para obtener la sesión y el cliente pasa al estado *READY*.

- **PLAY:**

Una vez en el estado *READY*, el cliente puede ejecutar la acción *PLAY* para enviar una petición de mismo nombre al servidor, informándole a este de que debe comenzar el envío de información multimedia, pasando al estado *PLAYING*, en el que el cliente reproduce el vídeo a medida que el servidor lo envía.

- **PAUSE:**

Ya en el estado *PLAYING*, el cliente puede enviar una petición *PAUSE* al servidor, informando de que debe detener el envío de información multimedia. De forma simultánea, se pausa en el cliente la reproducción al pasar al estado *READY*.

- **TEARDOWN:**

Estando el cliente en los estados *READY* o *PLAYING*, puede enviar una petición de *TEARDOWN* al servidor, con el fin de terminar la conexión y finalizar el cliente, volviendo al estado *INIT* y cerrando el reproductor.

En todas las operaciones a partir del *SETUP*, es preciso incluir la identificación de la sesión para que el servidor ejecute una respuesta.

Todas las comunicaciones *RTSP* se realizan en texto plano a través del socket establecido, al inicio del programa, mediante los parámetros de lanzamiento del cliente y servidor. Una interacción normal *RTSP* entre el cliente y servidor sería como la que se muestra a continuación en la *Figura 6*, donde “C:” y “S:” indican el texto enviado por el cliente y servidor, respectivamente, y no se incluye en la comunicación real.

```
C: SETUP movie.Mjpeg RTSP/1.0
C: CSeq: 1
C: Transport: RTP/UDP; client_port= 25000

S: RTSP/1.0 200 OK
S: CSeq: 1
S: Session: 123456

C: PLAY movie.Mjpeg RTSP/1.0
C: CSeq: 2
C: Session: 123456

S: RTSP/1.0 200 OK
S: CSeq: 2
S: Session: 123456

C: PAUSE movie.Mjpeg RTSP/1.0
C: CSeq: 3
C: Session: 123456

S: RTSP/1.0 200 OK
S: CSeq: 3
S: Session: 123456

C: PLAY movie.Mjpeg RTSP/1.0
C: CSeq: 4
C: Session: 123456

S: RTSP/1.0 200 OK
S: CSeq: 4
S: Session: 123456

C: TEARDOWN movie.Mjpeg RTSP/1.0
C: CSeq: 5
C: Session: 123456

S: RTSP/1.0 200 OK
S: CSeq: 5
S: Session: 123456
```

Figura 6. Ejemplo de comunicación RTSP entre cliente y servidor

Como se aprecia en la *Figura 6*, el servidor se limita a confirmar las peticiones recibidas, informando al cliente de que han sido correctamente procesadas. Cada operación debe ir acompañada de las cabeceras necesarias con los argumentos pertinentes al flujo y conexión activos.

La primera cabecera indica el tipo de operación solicitada, el flujo al que se refiere y el uso del protocolo *RTSP*. A continuación, se incluye una segunda cabecera con el número de secuencia que identifica la petición. Por último, una tercera cabecera especifica el identificador de sesión, asignado por el servidor durante la fase de *SETUP*.

La tercera cabecera únicamente toma un valor diferente en la fase *SETUP*, en la que el cliente debe especificar el protocolo y el puerto que usará para la conexión, de forma que el servidor pueda establecer la conexión y asignar al cliente el identificador de sesión para las operaciones posteriores.

6.2.3 Gestión del flujo *RTP*

Con el objetivo de obtener una reproducción fluida, el proyecto base implementa un servidor, encargado de enviar de forma constante y periódica el siguiente *frame* de vídeo, o *chunk* de audio, para que el cliente lo reproduzca tan pronto como lo reciba, determinando de esta forma la velocidad de reproducción.

El transporte de los datos multimedia se realiza mediante paquetes *RTP*, a su vez encapsulados en paquetes *UDP* que se encargan del envío de la información por la red. El uso de *RTP* permite la detección de pérdidas, seguimiento de tiempos y control del tipo de carga contenida en el paquete, entre otras funcionalidades relevantes para la transmisión de flujos multimedia.

Para el envío del flujo *RTP*, se establece previamente una conexión *UDP* cliente-servidor, esto empleando la información de sesión establecida mediante *RTSP* durante la fase de *SETUP*, siguiendo el proceso descrito en el anterior apartado 6.2.2. El servidor envía periódicamente cada pieza de información multimedia empaquetada y, es el cliente entonces, quien se encarga de desempaquetar el paquete *UDP* e interpretar la información contenida en el paquete *RTP*, así como convertir la carga alojada en un fotograma o audio reproducible para el usuario.

Para el envío periódico de la información multimedia, el servidor hace uso de un objeto *Timer* con el tiempo de duración del *frame* o *chunk*. El cliente, por otra parte, escucha activamente la conexión a la espera de nuevos paquetes multimedia, mediante otro *Timer*, aunque con una frecuencia más alta para evitar la pérdida de paquetes. El uso de dichos *timers* permite que el envío y reproducción mantengan un ritmo constante.

Es importante indicar que las especificaciones del protocolo *RTP* son deliberadamente incompletas en el estándar oficial, por lo que no especifica algoritmos de reproducción multimedia, regeneración de tiempos, sincronización entre flujos multimedia, ocultación o corrección de errores ni control de congestión [20, 23].

Esta implementación, aunque suficiente para el objetivo original de la práctica, presenta una evidente falla a la hora de exigir una velocidad estable de reproducción, además de resultar indispensable una reestructuración completa al escalar la complejidad del proyecto e implementar la simultaneidad del audio y vídeo.

6.2.4 Paquetización del contenido multimedia

Para lograr la transmisión del contenido multimedia, la información del vídeo y audio se divide en paquetes *RTP*, con la información necesaria para identificar la conexión y el paquete.

Esta división del contenido multimedia se establece en un único *frame* o fotograma por cada paquete *RTP*, en el caso del vídeo, y de un único *chunk* para el caso del audio, siendo un *chunk* un tiempo de audio determinado, por ejemplo, 100ms de contenido de audio, ajustado al contenido multimedia como el preparado específicamente para la práctica.

Con el objetivo de facilitar la lectura de la información multimedia para el proyecto base, los ficheros multimedia se codifican como la sucesión de los *frames* o *chunks* en bytes y, en el caso del vídeo, precedido cada *frame* con un atributo que determina la longitud en bytes del *frame*. El audio, en el caso del contenido usado en el proyecto, se codifica con una frecuencia de audio de 44100Hz, profundidad de bit de 16 y un único canal, resultando la longitud de *chunk* en un valor estático definido por la *Figura 7*.

$$\frac{\text{sampleRate} \times \left(\frac{\text{bitDepth}}{8} \right) \times \text{channelCount}}{10} \quad (1 \text{ paquete cada } 0,1 \text{ sec})$$

Figura 7. Ecuación de longitud de chunk de audio

Al extraer la información multimedia del fichero, cada *frame* o *chunk* es empaquetado dentro de un paquete *RTP*, en su campo de carga o *payload*. La estructura de cabecera del paquete *RTP* empleada en el proyecto corresponde a la establecida según el estándar *RFC 1889*, siguiendo además las actualizaciones del protocolo de *RFC 3550* [20, 23], el cual se ilustra en la *Figura 8*.

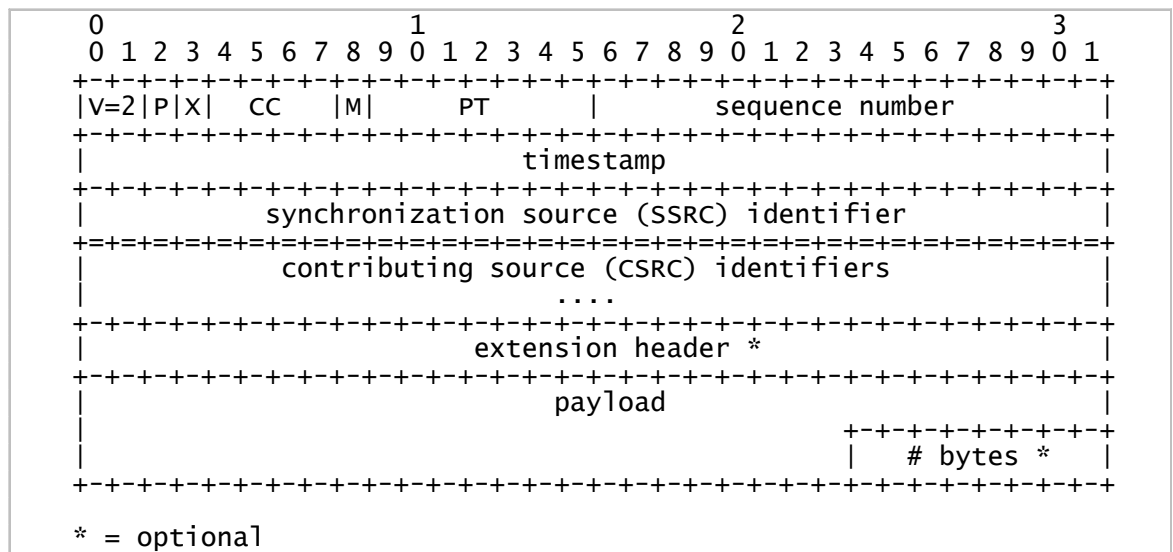


Figura 8. Cabecera RTP según RFC 3550

Los valores de cabecera empleados en el proyecto base fueron calculados de la siguiente forma:

- **Version (V) [2 bits]:** Corresponde a la versión de *RTP*, en este caso el valor empleado es siempre 2, indicando la versión de 2003.
- **Padding (P) [1 bit]:** Indica que el último octeto del paquete contiene el número de bytes que deben ser ignorados. Esto se emplea normalmente para encriptación u otros protocolos, aunque para este proyecto no es usado, por lo que siempre vale cero.
- **Extension (X) [1 bit]:** Determina si existe una extensión de cabecera tras la cabecera principal. Esta es una función raramente usada, ya que normalmente las extensiones suelen incluirse en la carga del paquete y, dado que no se emplean extensiones en esta implementación, su valor es siempre cero.
- **CSRC Count (CC) [4 bits]:** Especifica el número de identificadores *CSRC* que suceden a la cabecera, para esta implementación no es necesario su uso, por lo que su valor es siempre cero.
- **Marker (M) [1 bit]:** Permite el marcado de paquetes, aunque la interpretación del marcador queda abierta para la aplicación. Se suele emplear para permitir eventos significativos, aunque no se emplea el marcado en esta implementación, así que su valor permanece en cero.
- **Payload Type (PT) [7 bits]:** Indica el formato del *payload* contenido en el paquete, determinado por la aplicación e indispensable para discernir entre el audio y el vídeo en este proyecto. En esta implementación base, su valor será 26, en caso de ser un *frame* de *MJPEG*, o cero en el caso de contener un *chunk* de audio en bruto o *RAW*.
- **Sequence Number [16 bits]:** Permite conocer el número de paquete mediante un identificador único, incrementado con cada paquete *RTP* enviado. Usualmente este número comienza por un valor aleatorio para mayor seguridad y se asume un reinicio de la sesión si el número se encuentra a más de cien unidades fuera de orden.
- **Timestamp [32 bits]:** Refleja el instante de tiempo en el que fue creado el paquete. No precisa de usar la hora real y permite la medición del *jitter* y la sincronización de medios. Cada sesión *RTP* puede tener su propio reloj.
- **SSRC [32 bits]:** Identifica la fuente de sincronización, aunque para cumplir con los objetivos del proyecto no resulta necesario este atributo, por lo que siempre se considera cero.

Finalmente, dado que se ha establecido el número de *CSRC* a cero, no es necesario añadir ningún identificador *CSRC*, aunque en la práctica podría alojar una lista de hasta 15 identificadores. De igual manera, aunque a continuación se insertaría la extensión de cabecera, dado que no se emplea en esta implementación, el bit *E* se establece a cero y no se incluye la extensión.

Finalmente, una vez definida la cabecera *RTP*, los datos multimedia (o *payload*) son insertados en los bits inmediatamente posteriores, sin necesidad de especificar la longitud de estos.

6.3 Proyecto completo

Una vez completado el análisis y desarrollo del proyecto básico, se puede comenzar a construir sobre este la implementación de las técnicas de *buffering* y recuperación de pérdidas.

Las técnicas elegidas a desarrollar en la implementación constaron del desarrollo del *buffering* para la supresión del *jitter*, o un *de-jitter buffer*, y la recuperación de pérdidas de vídeo mediante *Forward Error Correction* (o *FEC*). Aunque para la implementación de dichas técnicas, era necesario mejorar el proyecto para una mayor fiabilidad, rendimiento y, en definitiva, una mejor base a la hora de implementar dichas funcionalidades.

En los siguientes apartados, se describe el proceso de desarrollo del proyecto completo, además del funcionamiento general del programa. La estructura del proyecto final viene compuesta por el siguiente diagrama de clases de la *Figura 9*.

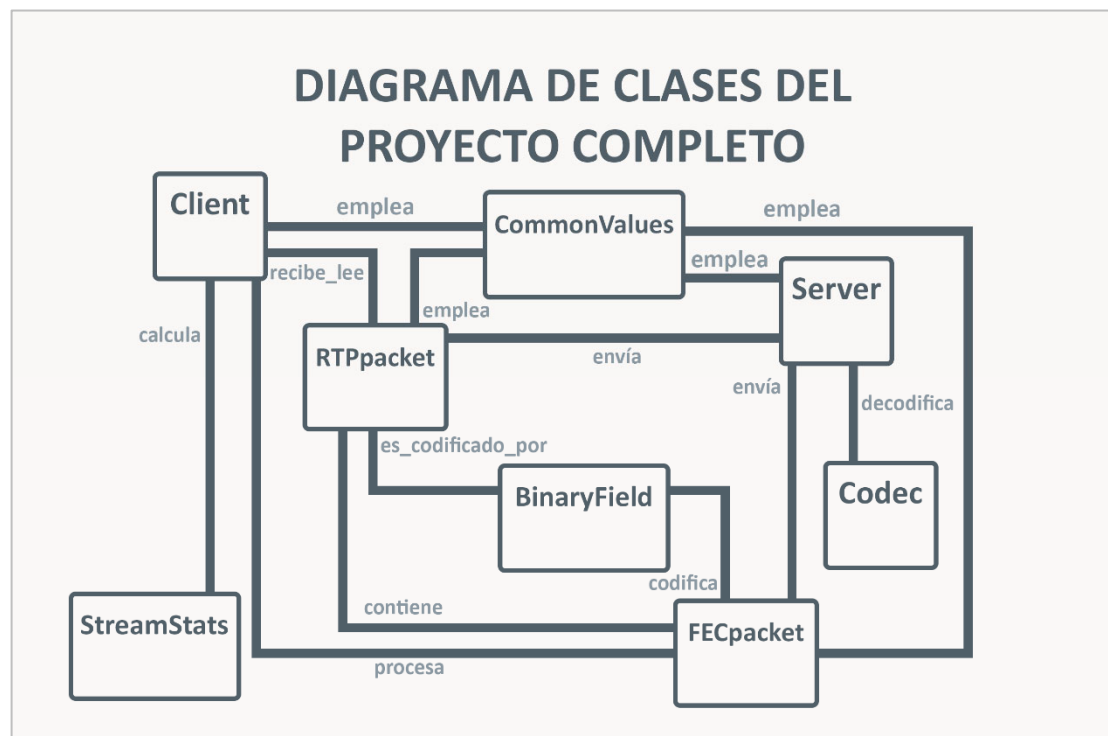


Figura 9. Diagrama de clases del proyecto completo

6.3.1 Códec avanzado

El comienzo de las mejoras del proyecto vino marcado por la implementación del códec avanzado, habilitando la reproducción simultánea de vídeo y audio, esto permitía realizar las pruebas sobre un entorno más realista, donde conviviesen ambos flujos.

El códec empleado se basó en la previa implementación, dada por parte del tutor para la asignatura de donde el proyecto original procede. En dicho códec, basado en *MJPEG*, tras una cabecera sencilla se codifican de forma alterna los *frames* de vídeo con los *chunks* de audio (véase *Figura 10*), siguiendo las especificaciones de cada uno, previamente empleadas en el proyecto base. Esto pretendía facilitar la lectura del fichero e incentivar el envío simultáneo de vídeo y audio desde el mismo hilo.



Figura 10. Codificación de fichero multimedia según el códec avanzado

El fichero comienza con una cabecera de 21 bytes con los siguientes atributos:

- **Versión (1 byte):** Especifica la versión de
- **FPS (1 byte):** Establece el número de fotogramas por segundo a los que se debería reproducir el vídeo.
- **Número de frames (4 bytes):** Indica la longitud del vídeo en fotogramas.
- **Ancho (4 bytes):** Define el ancho de los fotogramas del vídeo.
- **Alto (4 bytes):** Define el alto de los fotogramas del vídeo.
- **Número de pistas de audio (1 byte):** Especifica el número de pistas de audio asociadas al vídeo.
- **Frecuencia de muestreo (4 bytes):** Indica la tasa de muestreo del audio.
- **Profundidad de bit (1 byte):** Establece la profundidad de bits del canal de audio.
- **Número de canales (1 byte):** Especifica el número de canales de audio del fichero.

Aplicado a la implementación, el procesado de esta cabecera puede realizarse fácilmente, como se muestra en la *Figura 11*, mediante la lectura directa de los bits para, posteriormente, traducirse a enteros que puedan ser utilizados.

```
public void readHeader() throws Exception {  
  
    byte[] _Version = new byte[1];  
    byte[] _fps = new byte[1];  
    byte[] _numFrames = new byte[4];  
    byte[] _width = new byte[4];  
    byte[] _heigh = new byte[4];  
    byte[] _nAudioTracks = new byte[1];  
    byte[] _SamplingRate = new byte[4];  
    byte[] _bitDepth = new byte[1];  
    byte[] _channelCount = new byte[1];  
  
    fis.read(_Version, 0, 1);  
    Version = _Version[0];  
  
    fis.read(_fps, 0, 1);  
    fps = _fps[0];  
  
    fis.read(_numFrames, 0, 4);  
    numFrames = this.ByteArraytoInt(_numFrames);  
  
    fis.read(_width, 0, 4);  
    width = this.ByteArraytoInt(_width);  
  
    fis.read(_heigh, 0, 4);  
    heigh = this.ByteArraytoInt(_heigh);  
  
    fis.read(_nAudioTracks, 0, 1);  
    nAudioTracks = _nAudioTracks[0];  
  
    fis.read(_SamplingRate, 0, 4);  
    SamplingRate = this.ByteArraytoInt(_SamplingRate);  
  
    fis.read(_bitDepth, 0, 1);  
    bitDepth = _bitDepth[0];  
  
    fis.read(_channelCount, 0, 1);  
    channelCount = _channelCount[0];  
  
}
```

Figura 11. Bloque de código para procesar la cabecera del nuevo códec

Por otra parte, la información multimedia se encuentra en el fichero alternando los bits de *frames* de vídeo con bits de *chunks* de audio, precediendo cada *frame* cinco bytes para almacenar su longitud en bytes. En cuanto a los *chunks* de audio, no es necesario reservar espacio para almacenar su longitud, dado que tienen un tamaño constante determinado por la fórmula previamente mostrada en la *Figura 7*.

6.3.2 Envío de información multimedia

Inicialmente, el envío de vídeo y audio para la expansión del proyecto fue implementado en el mismo hilo, con el fin de que se enviaran simultáneamente ambos flujos. Sin embargo, el procesar el vídeo y audio en el mismo hilo, suponía un incremento significativo en el retardo de procesamiento, provocando un envío tardío de los paquetes y resultando especialmente notorio en el lado del cliente, además de ser una clara limitación para el contenido cuya frecuencia de audio y vídeo no coincidiera.

Para solucionar esto, se separó el audio y vídeo en dos flujos distintos, cada uno con su propia frecuencia y *socket*. En esencia, se tratan de dos hilos independientes del principal, donde cada uno abre el fichero a reproducir de forma independiente y extrae únicamente la información que es relevante para su flujo, omitiendo la información innecesaria. Es decir, el flujo de vídeo abre el fichero con su propio *FileInputStream* y lee cada *frame* de vídeo, omitiendo los *chunks* de audio que se encuentran intercalados en el fichero y, de igual forma, el flujo de audio extrae únicamente los *chunks* de audio del mismo fichero. Para realizar esto, se mantiene en cada hilo una variable que indica si los siguientes bytes corresponden a audio o vídeo, entonces se calcula cuántos bytes se deben dejar pasar para saltar el contenido no deseado. En el caso del vídeo, se calcula el número de bytes de audio mediante la fórmula de la *Figura 7*, mientras que, para el flujo de audio, simplemente se leen los bytes iniciales del *frame* de vídeo, correspondientes al largo en bytes de este.

En esta solución, la lectura simultánea del fichero permite a cada hilo leer de forma independiente la información que debe procesar al ritmo deseado, sin necesidad de almacenar en un *buffer* la información del hilo más lento hasta que se procese, lo que podría suponer una gran cantidad de información en ficheros más largos con distintas frecuencias.

Finalmente, tras leer el *frame* o *chunk* correspondiente, el hilo inserta la cadena de bytes en la carga de un paquete *RTP* con la información correspondiente, lo envía entonces al cliente antes de volver a esperar a enviar el siguiente trozo de información.

Ya hecha la separación de flujos, resulta en un servidor con dos hilos independientes, que procesan cada tipo de información y la insertan en paquetes *RTP* a un ritmo constante mediante interrupciones *sleep*.

6.3.3 Recepción de la información multimedia

De forma análoga, en el cliente se establecen tres hilos distintos en el momento de la reproducción: un hilo encargado de recibir la información *RTP* de la red, y dos hilos que procesan la información multimedia y la reproducen a la frecuencia adecuada, como se muestra en la *Figura 12*.

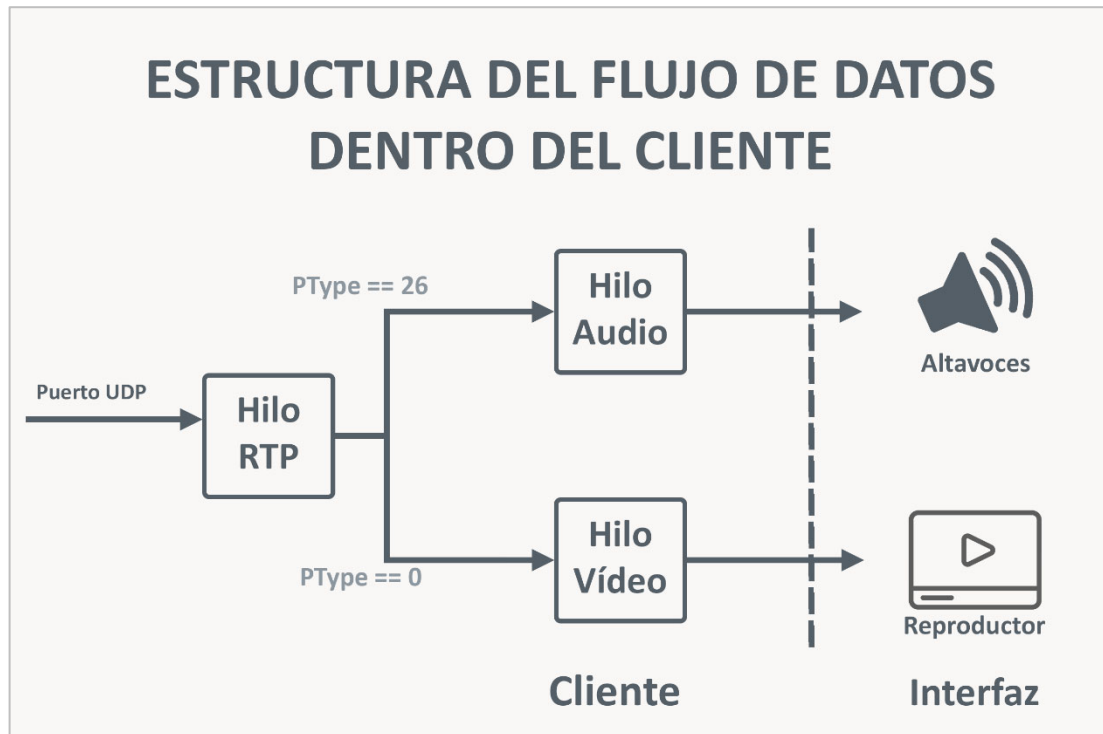


Figura 12. Estructura del flujo de datos dentro del cliente

El primer hilo, mostrado en el código de la *Figura 13*, se trata únicamente de una espera activa sobre el *socket RTP*, que recibe cada paquete *RTP* enviado por el servidor, sin importar el flujo de datos del que provenga, para posteriormente ser leído y, según el tipo de información multimedia que contenga, ser enviado a uno de los dos hilos de procesamiento para su posterior reproducción.

```
/** RTPSocketListener class pseudocode
 *
 * Thread to listen for incoming RTP packets on the RTP socket.
 */
class RTPSocketListener implements Runnable {
    public void run() {
        // Keep listening to the RTP socket until the state is INIT or the thread is
        // interrupted (End of program)
        while (state != Client.INIT && !Thread.currentThread().isInterrupted()) {
            // If player is paused ignore timeouts and keep waiting for the state to change
            if (state == Client.PLAYING) {
                RTPpacket rtp_packet;
                // Block until a packet is received
                RTPsocket.receive(rtp_packet);

                // Send packet to audio or video thread
                if (rtp_packet.getPayloadType() == RAW_TYPE) {
                    audioBuffer.offer(rtp_packet);

                } else if (rtp_packet.getPayloadType() == MJPEG_TYPE) {
                    videoBuffer.offer(rtp_packet);

                } else {
                    System.out.println("Unknown payload type");
                }
            }
        }
    }
}
```

Figura 13. Extracto del hilo de recepción RTP del cliente extendido

En cuanto al funcionamiento de los hilos de procesamiento de vídeo y audio, ambos tienen un comportamiento similar. Ambos hilos, siguiendo una frecuencia específica, establecida por la velocidad de reproducción del contenido, toman los fotogramas o *chunks* de audio que reciben del hilo *RTP* y extraen el contenido multimedia del paquete, para posteriormente reproducirlo en la interfaz del cliente. Con el fin de evitar desincronizaciones por pérdidas en la transmisión, el hilo se encarga de llevar la cuenta del número de secuencia correspondiente en cada momento, posponiendo aquellos paquetes que aún no deben procesarse.

En el caso del vídeo, como se puede ver en el extracto de la *Figura 14*, se crea una imagen a partir del fotograma e, invocando al hilo de la interfaz de forma asíncrona, se sustituye la imagen actual del reproductor por la nueva imagen creada. Aplicando la misma secuencia, es fácilmente trasladable al procesamiento de audio, donde se extrae la información multimedia y se envía al dispositivo de audio predeterminado.

```
/** videoTimerListener class pseudocode
 * Thread to process video frames from the video buffer.
 * Displays the frames in the GUI.
 */
class videoTimerListener implements Runnable {
    public void run() {
        while (running) { // Run until teardown
            synchronized (pauseLock) {
                while (paused && running) { // Check if video is paused
                    pauseLock.wait(); // Wait until the pauseLock is notified
                }
            }
            if (!running)
                break; // Exit if the running flag is false
            RTPpacket rtp_packet = videoBuffer.peek(); // Non-Blocking call
            if (rtp_packet != null) {
                // Check if packet is the expected one or expected number is -1 (initial state)
                if (rtp_packet.getSequenceNumber() == expectedPacketNb
                    || expectedPacketNb == -1) {
                    // Remove the packet from buffer if it is the expected one
                    videoBuffer.remove(rtp_packet);

                    if (expectedPacketNb == -1) {
                        // If this is the first packet, set the initial packet number
                        initialPacketNb = rtp_packet.getSequenceNumber();
                    }
                    // Update buffer size stats (synchronized)
                    synchronized (bufferLock) {
                        bufferSize = videoBuffer.size();
                    }

                    // get the payload bitstream from the RTPpacket
                    byte[] payload = rtp_packet.getPayload();

                    // get an Image object from the payload bitstream
                    Image image = createImage(payload, 0, payload_length);

                    // Update last played packet number
                    lastPlayedPacketNb = rtp_packet.getSequenceNumber();

                    // display the image as an ImageIcon object
                    iconLabel.setIcon(new ImageIcon(image));

                } else if (rtp_packet.getSequenceNumber() < expectedPacketNb) {
                    // If the packet is older than the expected, discard it
                    videoBuffer.remove(rtp_packet);
                }
                // If packet time has not yet arrived, we simply wait for the next
                // frame period and check again
            }
            expectedPacketNb++; // Increment expected packet number

            // Sleep for the frame period
            Thread.sleep(CommonValues.PLAYBACK_FRAME_PERIOD);
        }
    }
}
```

Figura 14. Extracto del hilo de procesamiento del video recibido

6.4 Implementación de un de-jitter buffer

Con el objetivo de mostrar el funcionamiento de un *de-jitter buffer*, además de comprobar su efectividad a la hora de corregir los tiempos de llegada de los paquetes, se realizó una implementación sencilla que corrigiese el *jitter* tanto en el flujo de audio como en el de vídeo.

El funcionamiento de este *buffer* es sencillo, como se muestra en la *Figura 15*, simplemente aloja los paquetes recibidos previo a su reproducción hasta que sean útiles, o ya no sean necesarios. Para esto, se crearon en la parte del cliente dos estructuras de cola prioritaria, ordenadas por número de secuencia de paquete, cuya misión consiste en alojar los paquetes de cada flujo de forma ordenada, para que los hilos de reproducción puedan consumirlos al ritmo deseado. Estas colas, son alimentadas por el hilo *RTP*, mencionado previamente y mostrado en la *Figura 13* que, a medida que llegan los paquetes de cada tipo, son enviados a dichos *buffers* como paso intermedio entre el hilo *RTP* y los hilos de procesamiento multimedia.

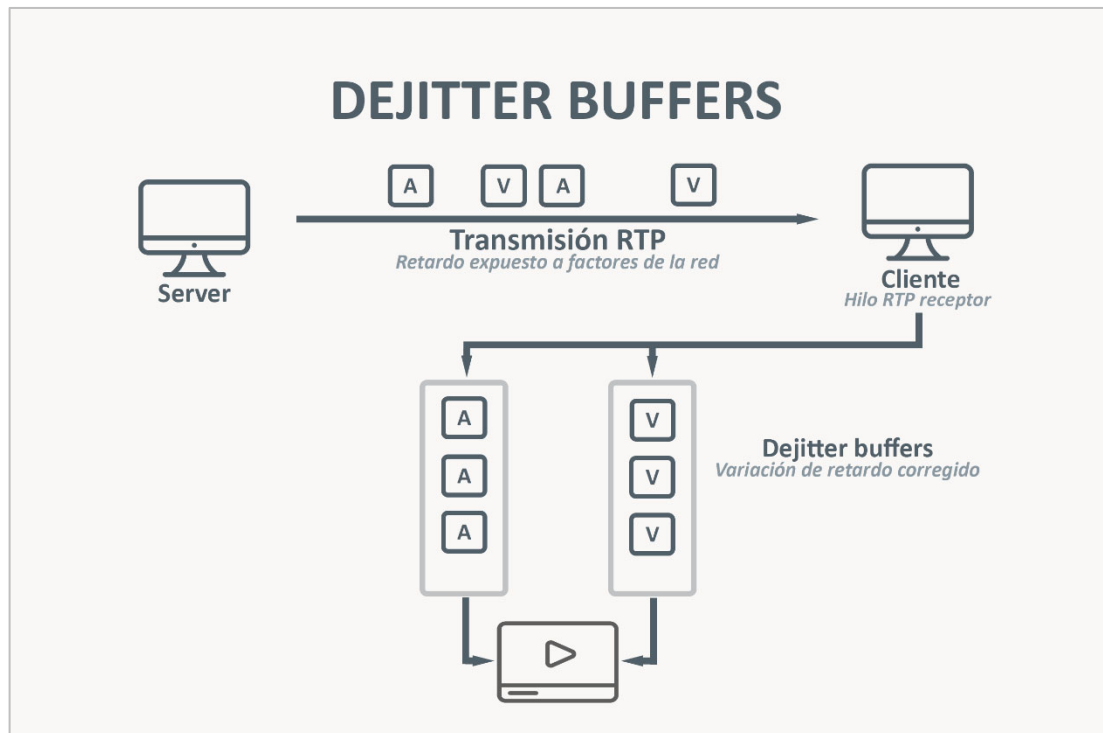


Figura 15. Funcionamiento de los de-jitter buffers

Gracias a la implementación de estos *buffers*, aunque los paquetes lleguen a distintos intervalos, se van rescatando por el hilo de procesamiento a una frecuencia fija, eliminando las pequeñas variaciones en el retardo por completo. Esto a su vez, conlleva un coste de procesamiento para mantener la cola y su eficiencia depende de la frecuencia de llegada de los paquetes, si estos tardan demasiado en llegar podría vaciarse el *buffer* por completo, pero si, por el contrario, el servidor envía demasiado rápido la información, la cola podría llegar a llenarse o, al menos, ocupar un tamaño excesivo en memoria.

Añadido a la función de corrección de *jitter*, al emplear colas prioritarias se corrige la llegada desordenada de paquetes, permitiendo además establecer un puente asíncrono entre el hilo de recepción *RTP* y los hilos de procesamiento.

La implementación de estos *buffers* puede apreciarse en los extractos ya mostrados en la *Figura 13* y *Figura 14*, donde el hilo *RTP* ofrece los paquetes recibidos a cada cola según el tipo de contenido, definido por el campo *PayloadType* del paquete *RTP*. Posteriormente, los paquetes son extraídos por los hilos multimedia, como se ve en el extracto de la *Figura 14*, en los que se solicita a las colas concurrentes el siguiente elemento mediante la función *poll*, bloqueando el hilo hasta que el siguiente paquete esté disponible o se produzca un *timeout* en el que se compruebe si el cliente ha pausado el vídeo. Mediante el uso de colas bloqueantes, es posible acceder de forma concurrente a la cola para extraer e insertar paquetes, evitando bloqueos, consumo innecesario de recursos y la corrupción de datos.

6.5 Recuperación de paquetes de vídeo mediante *FEC*

La implementación desarrollada a lo largo de este proyecto emplea la recuperación de paquetes para el flujo de vídeo mediante *Forward Error Correction*, permitiendo reconstruir un paquete cualquiera por cada grupo *FEC*.

Las bases de *Forward Error Correction* son sencillas, pues consiste en el envío periódico de un paquete especial, con redundancia de un número establecido de paquetes de datos. Existen múltiples formas de codificar dicha redundancia, aunque la empleada en este proyecto es mediante una operación *XOR* de los bytes de datos, también conocida como *XOR Parity*, y empleada en el *RFC 5109* para la implementación de *FEC* sobre *RTP* [30].

De esta forma, cada cierto número establecido de paquetes se genera un paquete de redundancia, que contiene el resultado de aplicar la operación *XOR* sobre los datos de todos los paquetes, de esta forma, es posible recuperar uno si se diera la pérdida o corrupción de esos datos, aplicando de nuevo la operación *XOR* sobre el resto de los paquetes, junto con el paquete de redundancia. Esta operación resulta en los datos del paquete perdido. En el artículo *RFC 5109* se ilustra mediante la *Figura 16*.

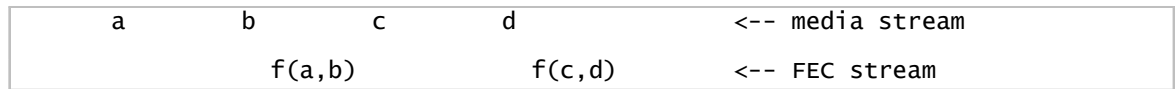


Figura 16. Diagrama del flujo FEC en RTP según RFC 5109

Sin embargo, este tipo de recuperación de datos cuenta con una limitación, solo es posible recuperar la información si se pierde un único paquete. Si dos paquetes o más se corrompen o pierden, incluido el paquete de redundancia, la recuperación de datos no es posible, declarando dicha información como perdida.

6.5.1 Estructura de los paquetes *FEC*

Lo primero para poder recuperar paquetes es definir cómo serán recuperados, en este caso, el paquete con la información *FEC* es el resultado de la operación *XOR* bit a bit sobre los paquetes de datos a proteger. Aunque el proyecto soporta la protección de hasta dieciséis paquetes de vídeo por el tamaño de máscara, se ha establecido por defecto que los grupos *FEC* empleados son de cinco paquetes, dado que grupos tan grandes no suponen una protección suficiente para este caso. Aunque este tamaño es modificable, como se mostrará más adelante.

Siguiendo la definición de la cabecera de primer nivel establecida por el estándar *RFC 5109* [30], se establecen a cero las *flags* de la cabecera de primer nivel, inmediatamente después, el *PayloadType Recovery* como el resultado de la operación *XOR* del *PayloadType* de todos los paquetes del grupo. Seguido de esto, en los dieciséis bits siguientes, se inserta el número de secuencia base, es decir, el número de secuencia más bajo del grupo *FEC*, es decir, el del paquete más antiguo protegido por este grupo. Sobre este número se calculará más tarde la máscara en la cabecera de segundo nivel.

De forma similar al *PayloadType Recovery*, el campo de treintaidós bits destinado a la recuperación del *timestamp*, así como el campo posterior a este de dieciséis bits, reservado para la recuperación de la longitud de paquete, se codifican con el resultado de aplicar la operación *XOR* sobre los campos *timestamp* y de longitud de todos los paquetes del grupo, respectivamente, dando por finalizada así, la definición de la cabecera de primer nivel del paquete de *Forward Error Correction*, como se ilustra en el esquema de la *Figura 17*.

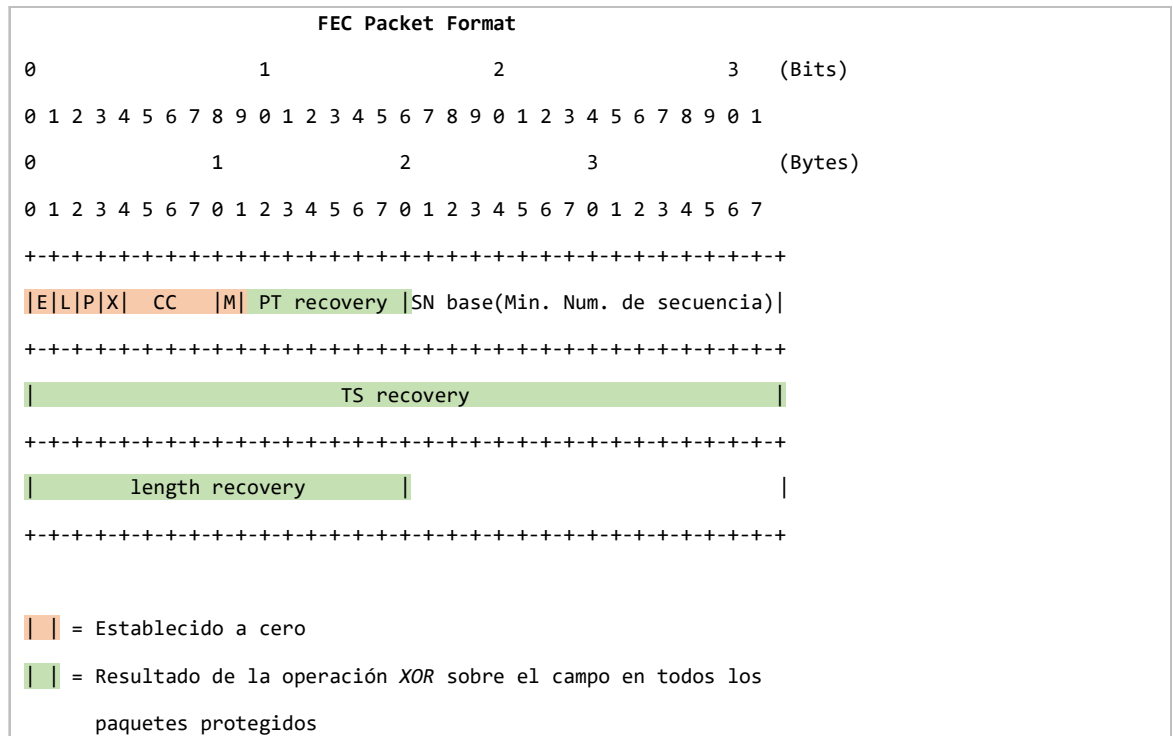


Figura 17. Esquema con la definición de los campos de la cabecera de primer nivel FEC

Inmediatamente seguido de la cabecera de primer nivel, se define la cabecera de segundo nivel, siguiendo el esquema de la *Figura 18* [30].

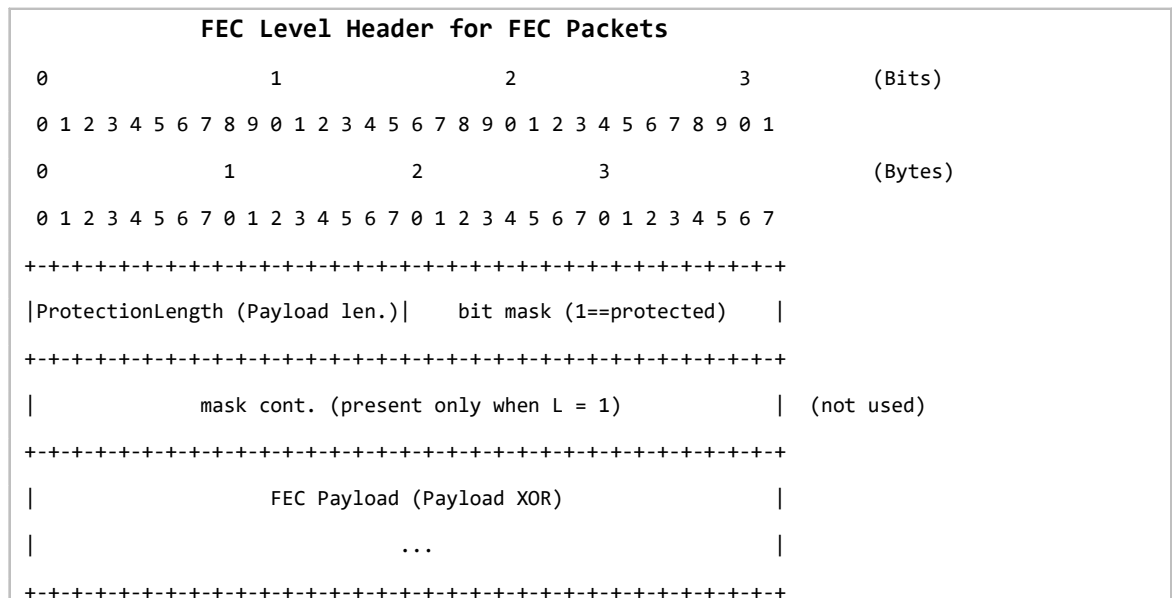


Figura 18. Esquema con la definición de los campos de la cabecera de segundo nivel FEC

La cabecera comienza por definir la máscara de bits, que indica los paquetes protegidos en el grupo. Esta se define mediante un array de dieciséis bits, en los que cada bit representa un paquete. Cada bit representa el número de secuencia siguiente, es decir, la suma de número de secuencia base, sumada al número de posiciones desde la izquierda, resulta en el número de secuencia de dicho paquete del grupo. Este cálculo es representado en la ecuación de la *Figura 19*.

$$SeqNum = BaseSeqNum + BitPosition$$

Figura 19. Cálculo del número de secuencia de un paquete protegido según máscara de bits FEC

A pesar de que es posible habilitar una extensión de esta máscara de hasta treintaidós bits, este campo no es usado en la implementación, dado que no resulta relevante emplear grupos tan grandes para la aplicación actual, por lo que el campo es omitido y su correspondiente *flag*, establecido a cero.

Por último, el resto de los bits del final del paquete alojan el resultado de aplicar la operación *XOR* sobre la carga o *payload* de los paquetes del grupo, lo cual a su vez determina el campo *ProtectionLength*, alojado en los primeros dieciséis bits de la cabecera de segundo nivel y correspondiente a la longitud de la carga *FEC*. Es importante no confundir este campo con el pasado *Length Recovery*, que contiene la paridad del campo de longitud, ya que *ProtectionLength* únicamente declara el tamaño de la carga de redundancia.

6.5.2 Envío de paquetes *FEC* y recuperación de vídeo

Ya definida la estructura y el proceso de creación de paquetes de redundancia, queda establecer mecanismos de detección de pérdidas e implementar un protocolo de recuperación de los paquetes afectados.

El proyecto actualmente cuenta con protección de paquetes únicamente para el flujo de vídeo, aunque la estructura es fácilmente replicable, tanto para audio como para cualquier otro tipo de información.

Con el fin de manejar con precisión la información relativa a *FEC*, la implementación cuenta con su propio flujo reservado para los paquetes de recuperación, estableciendo en el cliente su propio hilo de procesamiento, ilustrado en la *Figura 20*, tal como se ha hecho para los flujos de audio y vídeo.

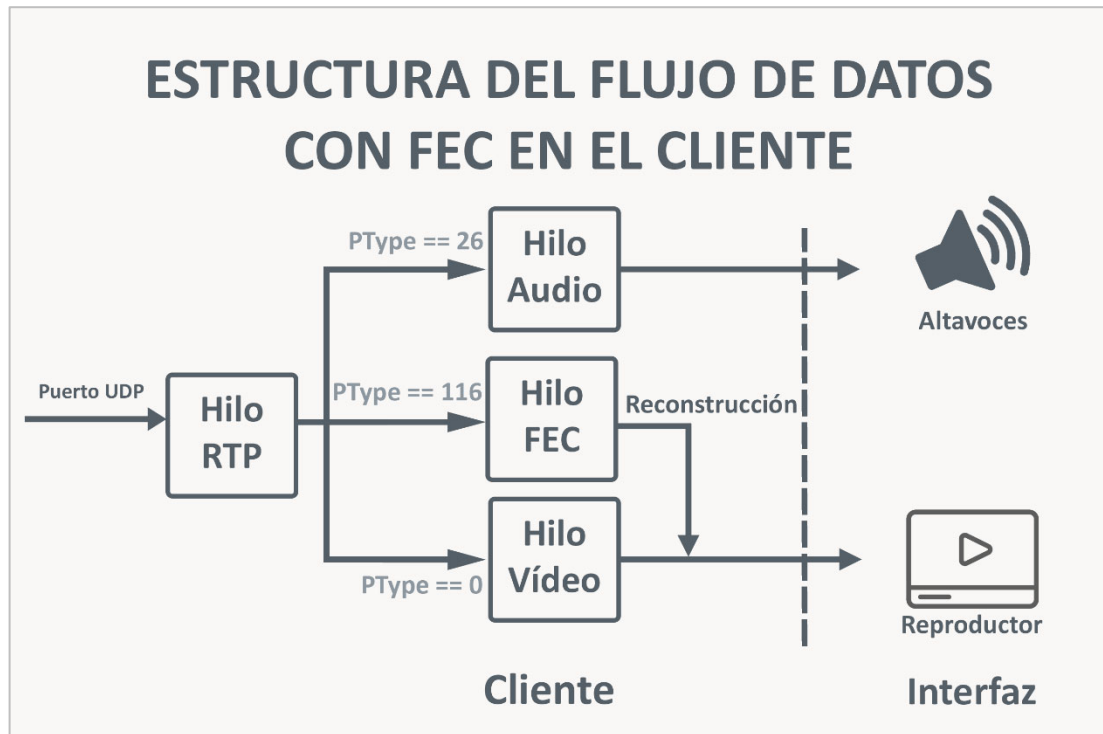


Figura 20. Estructura del flujo de datos con FEC

El inicio de la implementación de esta técnica comienza en el lado del servidor, donde cada vez que un paquete de vídeo es enviado al cliente, este es añadido a una lista representativa del grupo *FEC* actual. Dicho grupo *FEC* tiene un tamaño máximo fijo. Una vez el grupo llega al tamaño máximo establecido, o cuando se alcanza el final del archivo, se construye un nuevo paquete con la cabecera de recuperación, mostrada en la anterior *Figura 18*, y se codifica la información, según las especificaciones establecidas en el anterior punto 6.5.1. Tras esto, el paquete de redundancia es enviado al cliente y los paquetes del grupo *FEC* son liberados, dando paso a la formación del siguiente grupo.

En el lado del cliente, y de manera simultánea a la recepción de vídeo, el paquete de recuperación llega al hilo *RTP* donde, siguiendo su valor *PType*, es enviado al hilo *FEC* a través de una lista que actúa de *buffer*, de donde el hilo *FEC* va tomando los paquetes de recuperación para su procesamiento.

Con el fin de poder ser usados para recuperar paquetes futuros del mismo grupo, el cliente mantiene en una lista de protección los paquetes de vídeo que llegan, siendo liberados cuando ya no son relevantes. Esto permite recuperar paquetes que, del mismo grupo, pudieran llegar después de que los más antiguos del grupo hayan sido reproducidos.

En caso de que todos los paquetes del grupo hayan llegado correctamente, el paquete de recuperación es descartado por el hilo de procesamiento, pasando al siguiente, de lo contrario, se toman todos los paquetes recibidos del grupo y comienza la recuperación. Sólo si un único paquete del grupo se ha perdido, se procede a la recuperación de este con la información del resto del grupo. Si el paquete de redundancia no es recibido, no se intenta la recuperación.

El *buffer* que contiene los paquetes protegidos, por defecto, limita su tamaño a 48 paquetes, esto se debe a que, dado que el cliente no conoce el tamaño del grupo que va a emplear el servidor, se establece un tamaño de *buffer* de tres veces el tamaño máximo de un grupo, siendo cada grupo máximo de dieciséis paquetes. De esta forma se limita la cantidad de memoria ocupada por el *buffer*, a la par que dando margen para que el hilo *FEC* pueda emplear los paquetes protegidos a tiempo, si así lo requiere.

La recuperación de la información es simple, solo se vuelven a aplicar las operaciones empleadas en la creación del paquete de redundancia, incluyendo esta vez dicho paquete en las operaciones. El resultado es la información del paquete extraviado, con la que fácilmente se reconstruye el paquete *RTP* original y se envía al *buffer* de vídeo, con la esperanza de ser reconstruido a tiempo para su reproducción. De lo contrario, el paquete reconstruido es descartado y marcado en las estadísticas como *LATE*, indicando que, aunque ha sido reconstruido exitosamente, no ha llegado a tiempo para su reproducción.

6.6 Mejoras en la interfaz

Con la implementación de nuevas funcionalidades y mejoras en el funcionamiento del cliente y servidor, también llegaron algunas mejoras a la interfaz. Estas modificaciones tuvieron, como objetivo principal, mejorar la legibilidad del proyecto y permitir un mejor seguimiento estadístico de la transmisión en tiempo real, dando una mejor base sobre la que poder realizar pruebas más adelante.

Aunque el reproductor original del cliente se mantuvo intacto, se renovó por completo la ventana auxiliar de estadísticas del cliente, todo ello para acomodar las nuevas capacidades del software, mostrada en la *Figura 21*. Esto permite un mejor seguimiento de las funcionalidades del cliente.

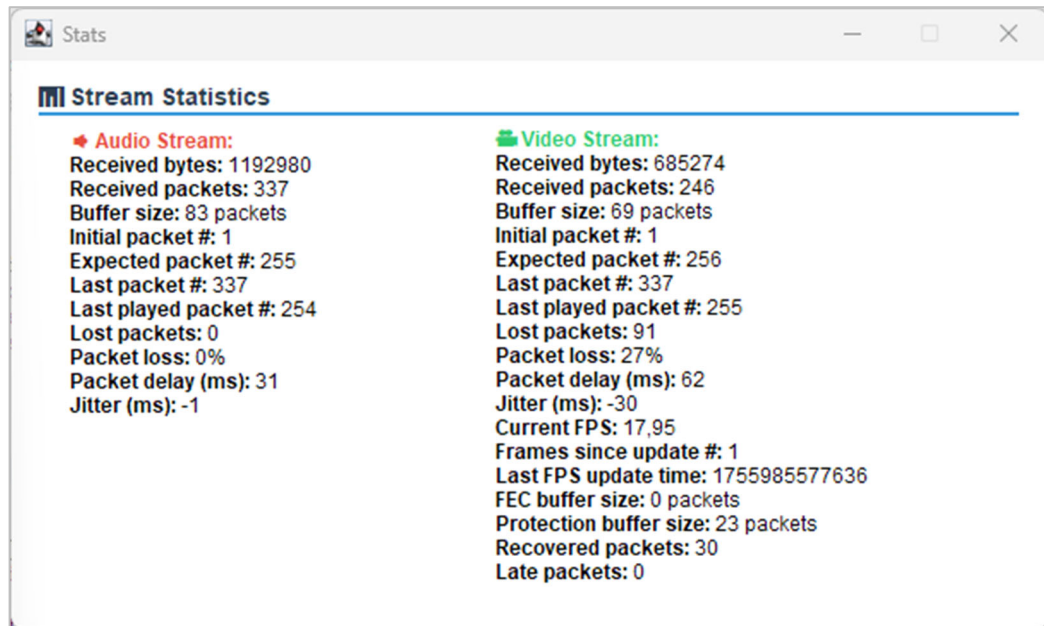


Figura 21. Interfaz renovada de estadísticas del cliente

En la *Figura 22* se muestra una comparación en la que, además de separar las estadísticas para ambos flujos de audio y vídeo, mejorar su presentación y disposición, gran cantidad de estadísticas relevantes fueron añadidas, aunque sobre estas estadísticas se entrará en más detalle más adelante. Añadido a esto, las estadísticas tienen ahora un pequeño retardo de actualización para paliar problemas de rendimiento, derivados de la actualización de estas sin límite establecido.

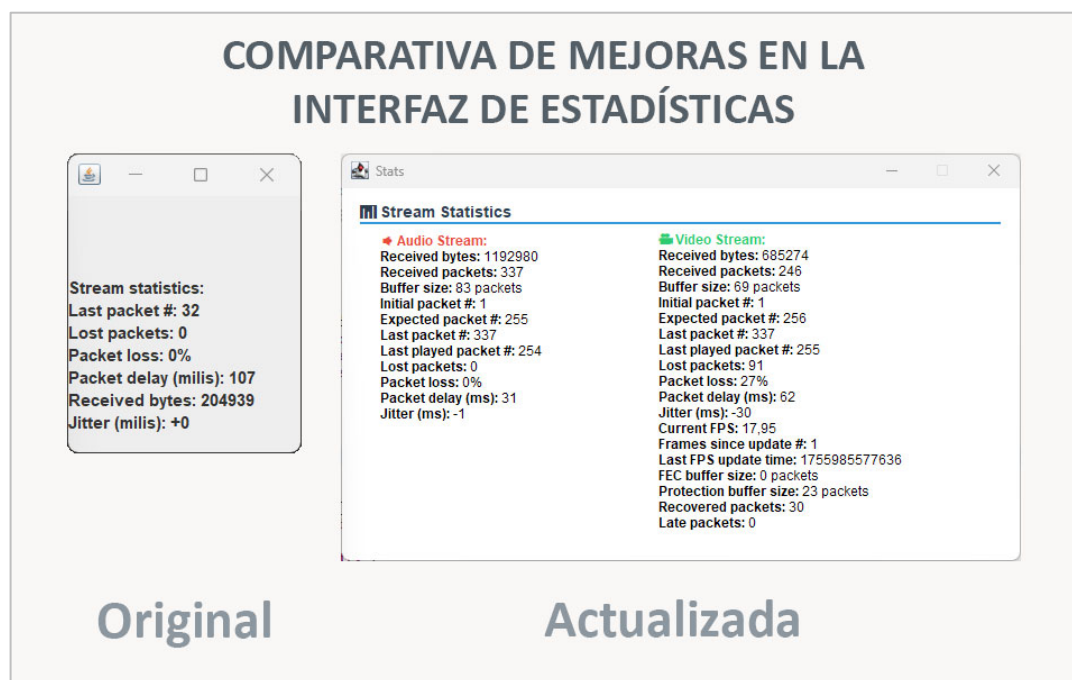


Figura 22. Comparativa de mejoras en la interfaz de estadísticas

Continuando con el cliente, una nueva ventana fue implementada con el fin de permitir la visualización del *buffer* de vídeo en tiempo real. La interfaz, la cual se muestra en la siguiente *Figura 23*, consta de un gráfico de barras en el que cada una representa un *frame*, los cuales a su vez mantienen su número de secuencia y su estado asociado, permitiendo identificar el paquete correspondiente con el estado en el que se encuentra.

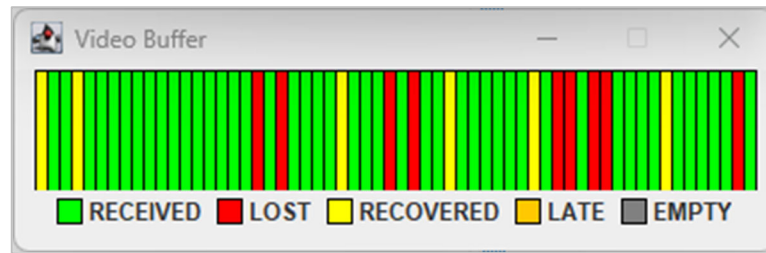


Figura 23. Gráfico de buffer de vídeo del cliente

El código de color, como se muestra en la leyenda de la parte inferior del gráfico, consta de los siguientes estados:

- **RECEIVED:** El paquete ha sido recibido correctamente y a tiempo para su reproducción.
- **LOST:** El paquete no ha llegado, por lo que se intentará su reconstrucción si es posible.
- **RECOVERED:** Mediante la técnica de *Forward Error Correction*, la información del paquete ha sido reconstruida exitosamente, a tiempo para su reproducción.
- **LATE:** Aunque la información del paquete ha sido recibida o reconstruida mediante *Forward Error Correction*, no ha llegado a tiempo para su reproducción, por lo que ha sido descartado.
- **EMPTY:** Estado vacío, únicamente aparece en el principio simbolizando las casillas que aún no han sido rellenas con estados, tras esto el gráfico se desplaza automáticamente por lo que no vuelve a aparecer este estado.

Esta información aparece también en la propia interfaz, mediante un mensaje flotante al mantener el cursor sobre cada elemento de la leyenda, de igual forma, mantener el cursor sobre los paquetes mostrados en el gráfico permite ver el número de secuencia asociado, habilitando un mejor análisis de lo que está sucediendo en la transmisión.

Finalmente, la última mejora de interfaz viene dada en el lado del servidor, donde se aprecia una completa remodelación de la apariencia y capacidades de la ventana de información, mostrada a continuación en la *Figura 24*.

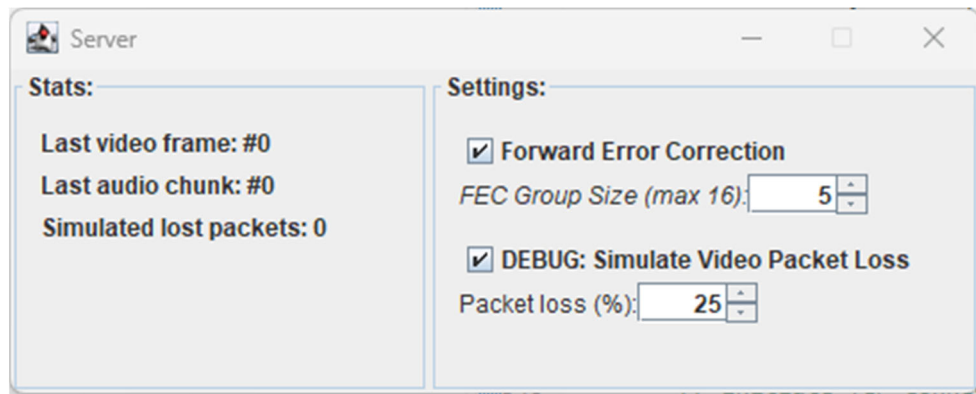


Figura 24. Interfaz renovada del servidor

Como se puede apreciar en la *Figura 24*, la interfaz ahora cuenta con una composición de dos paneles en el que, no solo se muestran las estadísticas de los paquetes enviados por cada flujo de datos, sino que también incluye un menú de opciones que permite habilitar, deshabilitar y modificar las funcionalidades, tanto de *Forward Error Correction*, como la simulación de pérdidas de paquetes de vídeo para entornos de prueba.

Modificar el número de *FEC Group Size* permite seleccionar el número de paquetes que deben formar un grupo *FEC*, permitiendo experimentar con distintos tamaños. Por otra parte, el simulador de pérdidas de paquetes de vídeo permite que el servidor no envíe un porcentaje de los *frames*, simulando su pérdida en el cliente.

El menú de opciones está disponible para su modificación hasta que se recibe la orden de *SETUP* desde el cliente, momento en el que el panel queda bloqueado a su modificación, tal y como se ve en la *Figura 25*.

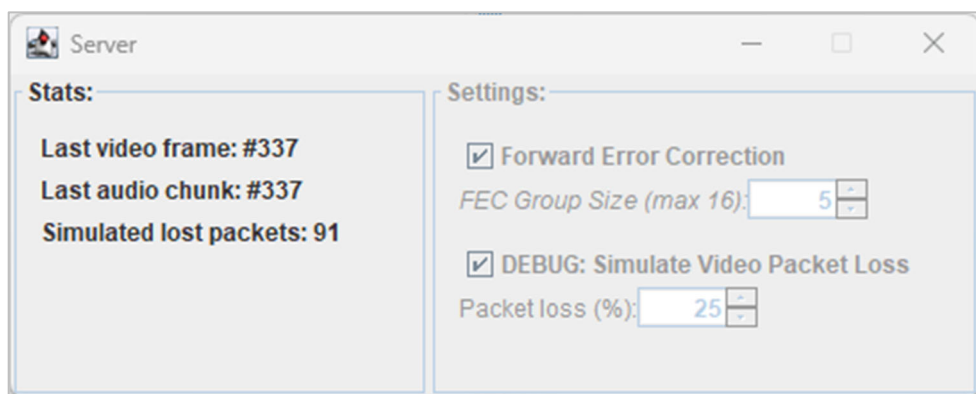


Figura 25. Menú de opciones del servidor bloqueado

Comparativamente, la interfaz del servidor presenta la modificación más notable, como se ve en la siguiente *Figura 26*. Frente a una interfaz bastante escasa con un contador único, se ha propuesto una interfaz más completa y ajustada a las capacidades del servidor, sumado a la implementación de opciones, que permiten una forma más práctica de realizar pruebas sobre el proyecto, sin necesidad de modificar el código original.

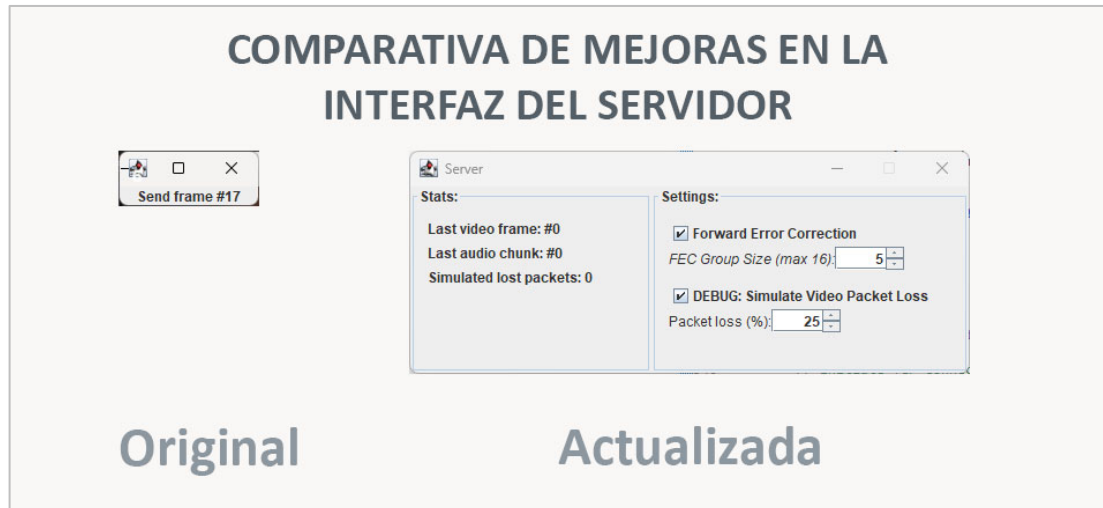


Figura 26. Comparativa de mejoras en la interfaz del servidor

6.7 Parámetros de lanzamiento

Con el objetivo de facilitar su despliegue durante las pruebas, tanto el cliente como el servidor cuentan con parámetros de lanzamiento, que permiten la habilitación y modificación de las funcionalidades del programa.

En el caso de los parámetros del cliente, mostrados en la *Figura 27*, la sintaxis apenas ha cambiado desde el proyecto original.

```
java Client <hostname> <RTSP listening port> <video file path> [-vv]
Parameters:
- hostname
  IP address or hostname of the RTSP server.
- RTSP listening port
  Port number of the RTSP server.
- video file path
  Path to the video file on the server.
- -v
  Enable verbose mode for detailed logging.
- -V
  Enable super verbose mode for extensive debugging output.
```

Figura 27. Parámetros de lanzamiento de cliente

Los parámetros originales *hostname*, *RTSP listening port* y *Video file path* se mantienen igual, permitiendo configurar la conexión con el servidor y solicitar el vídeo para su transmisión, mientras que las opciones adicionales *-v* y *-V* fueron añadidas con el fin de manejar la cantidad de información que debe ser mostrada por consola, permitiendo establecer un mayor nivel de detalle mediante el modo *Super Verbose*.

Por otra parte, los parámetros del servidor, como muestra la *Figura 28*, ahora cuentan con múltiples opciones, que permiten establecer las configuraciones del menú directamente durante arranque, mejorando la automatización y despliegue del servidor.

```
Usage: java Server <RTSP listening port> [-v] [-f=FEC group size]
[-s=Simulated packet loss]

Parameters:

<RTSP listening port> <dd>Port number for RTSP connection (e.g., 1025)

- -v
    Enable verbose mode for debugging output
- -f=groupSize
    Set FEC group size (between 2 and 16, default is enabled with a
    value of 5)
- -s=packetLoss
    Set simulated packet loss percentage (between 1 and 100, default is
    disabled with a value of 5)

Disables FEC or simulated packet loss by setting their values to 0.
```

Figura 28. Parámetros de lanzamiento de servidor

Manteniendo la primera posición para establecer el puerto *RTSP* del proyecto original, no solo se ha añadido un parámetro para establecer el detalle de la salida por consola, sino que dos parámetros adicionales de configuración fueron implementados. Tanto la opción de *FEC*, como la simulación de pérdidas de paquetes, pueden habilitarse y establecer sus valores mediante el parámetro con el valor y, estableciendo este a cero, la función queda deshabilitada. Por defecto, *FEC* viene habilitado con un grupo de cinco paquetes, mientras que la simulación de pérdidas de paquetes viene deshabilitada con un valor preestablecido a 5, aunque este valor deshabilitado solo afecta a cómo se muestra en el selector de la interfaz.

6.8 Estadísticas

Además de las mejoras en la funcionalidad del proyecto, resultó esencial la implementación de un seguimiento estadístico en tiempo real, uno más adecuado a las nuevas capacidades del software. Aunque la interfaz del servidor solo indica los paquetes enviados para vídeo y audio, el cliente cuenta con multitud de estadísticas con las que medir las cualidades de la transmisión:

- **Estadísticas comunes entre vídeo y audio**
 - ***Received bytes***: Número total de bytes recibidos durante la transmisión. Se mide de manera independiente para el flujo de audio y vídeo.
 - ***Received packets***: Cantidad total de paquetes recibidos. Disponible tanto en audio como en vídeo.
 - ***Buffer size***: Tamaño del *buffer* del flujo, expresado en número de paquetes. Se encuentra presente en ambos flujos.
 - ***Initial packet #***: Identificador del primer paquete recibido en la transmisión, medido de manera separada para audio y vídeo.
 - ***Expected packet #***: Número de secuencia del siguiente paquete que el sistema espera recibir. Se aplica en ambos flujos y se incrementa periódicamente al ritmo de la reproducción.
 - ***Last packet #***: Identificador del último paquete recibido, tanto en audio como en vídeo.
 - ***Last played packet #***: Número de secuencia del último paquete reproducido por el cliente en cada flujo.
 - ***Lost packets***: Cantidad de paquetes que no llegaron a recibirse correctamente. Se presenta en audio y en vídeo.
 - ***Packet loss (%)***: Porcentaje total de pérdida de paquetes. Disponible en ambos flujos.
 - ***Packet delay (ms)***: Retraso medio en la llegada de los paquetes, expresado en milisegundos. Se mide de manera independiente para audio y vídeo.
 - ***Jitter (ms)***: Variación del retardo en la llegada de los paquetes, indicador de estabilidad en la transmisión. Se calcula en ambos flujos.
- **Estadísticas específicas del flujo de vídeo**
 - ***Current FPS***: Tasa de fotogramas por segundo en el momento de la medición.
 - ***Frames since update #***: Número de fotogramas procesados desde la última actualización estadística.
 - ***Last FPS update time***: Instante temporal en el que se registró la última actualización de los fotogramas por segundo.
 - ***FEC buffer size***: Tamaño del *buffer* destinado a paquetes de redundancia para la corrección de errores (*Forward Error Correction*).
 - ***Protection buffer size***: Tamaño del *buffer* que aloja temporalmente los paquetes de vídeo, empleado para recuperación de datos con *Forward Error Correction*.
 - ***Recovered packets***: Número de paquetes que fueron recuperados mediante corrección de errores.
 - ***Late packets***: Cantidad de paquetes recibidos o recuperados demasiado tarde como para ser reproducidos.

Con el fin de no colapsar el hilo de la interfaz y mantener un rendimiento razonable, las estadísticas son actualizadas periódicamente cada 200 milisegundos.

7. RESULTADOS

Con la aplicación desarrollada, es momento de realizar pruebas para determinar la eficacia de las técnicas implementadas en mantener la calidad de experiencia del cliente. Para esto, se recopilaban datos de la misma transmisión bajo distintos escenarios emulados.

El contexto de ejecución de las pruebas se realizó dentro de una red emulada empleando la herramienta de código abierto *Mininet* [32], diseñada para la emulación de redes en una misma máquina. El uso de esta herramienta nos permite emular los distintos escenarios y cualidades que podría presentar la red por la que se transmite el flujo multimedia. Añadido a esta y, con el fin de mostrar gráficamente la red emulada, se hizo uso de *MiniNAM*, una interfaz visual escrita en Python de código abierto que permite mostrar no solo la estructura de la red, sino que también el tráfico en tiempo real [33, 34].

Además de esto, se empleó el uso de la conocida herramienta de análisis de tráfico *Wireshark* [35], que permitió extraer las estadísticas del tráfico desde un punto externo de la aplicación.

7.1 Topología de la red para las pruebas

Para que los resultados sean consistentes, se empleó en todas las simulaciones la misma topología de red emulada, mostrada en la *Figura 29* y consistente de dos subredes unidas por routers, esto permitiría añadir retardos y pérdidas de paquetes de forma más sencilla.

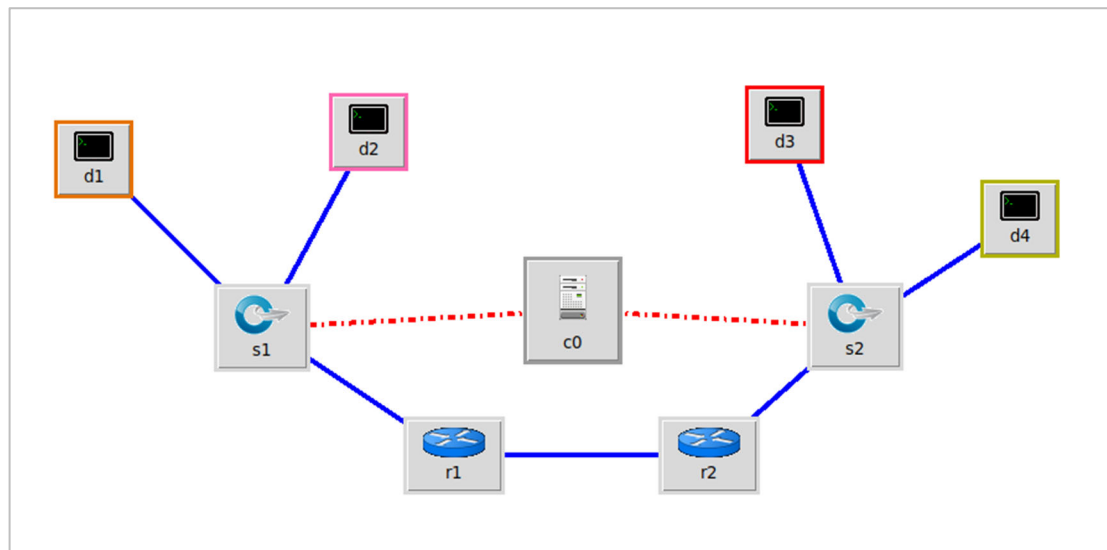


Figura 29. Topología de red de pruebas

Como se puede apreciar en la *Figura 29*, la red se compone de dos subredes, cada una con su propio *switch* y dos equipos. En la *Figura 30* a continuación se muestra la información de la red mostrada por *Mininet*.

```
<Host d1: d1-eth0:10.0.0.251 pid=1655>
<Host d2: d2-eth0:10.0.0.252 pid=1660>
<Host d3: d3-eth0:10.1.0.251 pid=1662>
<Host d4: d4-eth0:10.1.0.252 pid=1664>
<LinuxRouter r1: r1-eth1:10.0.0.1,r1-eth2:10.100.0.1 pid=1668>
<LinuxRouter r2: r2-eth1:10.1.0.1,r2-eth2:10.100.0.2 pid=1670>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None pid=1675>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None,s2-eth3:None pid=1678>
<Controller c0: 127.0.0.1:6653 pid=1647>
```

Figura 30. Información de topología Mininet

Con el fin de facilitar su despliegue, la topología se ejecuta mediante un script en Python similar al de la *Figura 31*, donde se definen los distintos equipos, routers, switches y enlaces.

```
# Add 2 routers in two different subnets
r1 = self.addHost('r1', cls=LinuxRouter, ip='10.0.0.1/24')
r2 = self.addHost('r2', cls=LinuxRouter, ip='10.1.0.1/24')

# Add 2 switches
s1 = self.addSwitch('s1', failmode='standalone')
s2 = self.addSwitch('s2', failmode='standalone')

# Add host-switch links in the same subnet
self.addLink(s1,
              r1,
              intfName2='r1-eth1',
              params2={'ip': '10.0.0.1/24'})

self.addLink(s2,
              r2,
              intfName2='r2-eth1',
              params2={'ip': '10.1.0.1/24'})

# Add router-router link in a new subnet for the router-router connection
self.addLink(r1,
              r2,
              intfName1='r1-eth2',
              intfName2='r2-eth2',
              params1={'ip': '10.100.0.1/24'},
              params2={'ip': '10.100.0.2/24'})

# Adding hosts specifying the default route
d1 = self.addHost(name='d1',
                  ip='10.0.0.251/24',
                  defaultRoute='via 10.0.0.1')
d2 = self.addHost(name='d2',
                  ip='10.0.0.252/24',
                  defaultRoute='via 10.0.0.1')

d3 = self.addHost(name='d3',
                  ip='10.1.0.251/24',
                  defaultRoute='via 10.1.0.1')
d4 = self.addHost(name='d4',
                  ip='10.1.0.252/24',
                  defaultRoute='via 10.1.0.1')

# Add host-switch links
self.addLink(d1, s1)
self.addLink(d2, s1)
self.addLink(d3, s2)
self.addLink(d4, s2)
```

Figura 31. Script de creación de la topología de pruebas

Con el fin de mantener una uniformidad en las pruebas, todas son ejecutadas asignando al host *d1* como el servidor y el host *d4* como cliente y aplicando las modificaciones a la red en el enlace entre ambos routers que actúan de paso entre las subredes.

7.2 Escenario 1: Recuperación de paquetes en una red con pérdidas del 10%

Para el primer escenario se preparó una red emulada, de misma topología que la mostrada en la *Figura 29*, en la que se introdujo un porcentaje de pérdidas del 10% esto, con el fin de comprobar la efectividad de la corrección de errores hacia adelante o *FEC* en entornos con pérdidas notorias.

Para la preparación de la topología, se modificó el script de la topología para que el enlace entre ambos routers incluyese una pérdida del 10% tal y como se destaca en la *Figura 32*.

```
# Add router-router link in a new subnet for the router-router
connection
self.addLink(r1,
              r2,
              intfName1='r1-eth2',
              intfName2='r2-eth2',
              params1={'ip': '10.100.0.1/24'},
              params2={'ip': '10.100.0.2/24'},
              loss=10)
```

Figura 32. Modificación de topología para pérdidas

Con la topología modificada, se lanza *Mininet* empleando *MiniNAM* y el fichero del script en Python mediante el comando de la *Figura 33*.

```
sudo python MiniNAM.py --custom topology.py --topo mytopo --link=tc
```

Figura 33. Comando de lanzamiento de la topología en Mininet

Ya lanzada la topología, es necesario configurar la ruta entre ambas subredes, para ello se emplean los siguientes comandos de la *Figura 34* en la terminal de *Mininet*.

```
mininet> r1 ip route add 10.1.0.0/24 via 10.100.0.2
mininet> r2 ip route add 10.0.0.0/24 via 10.100.0.1
```

Figura 34. Enrutamiento entre subredes en Mininet

Es posible ahora probar esta configuración mediante un *ping* entre equipos de las distintas subredes.

Finalmente, dado que la aplicación emplea paquetes de gran tamaño, al superar la *Maximum Transmission Unit* o *MTU*, los paquetes se fragmentan automáticamente, lo que puede provocar que en un escenario de pérdidas se pierdan fragmentos de un paquete, duplicando las pérdidas. Para esto, se configura el *MTU* de la red completa mediante la secuencia de comandos de la *Figura 35*.

```
# Set MTU to a high value to avoid packet fragmentation
d1 ifconfig d1-eth0 mtu 9000
d2 ifconfig d2-eth0 mtu 9000
d3 ifconfig d3-eth0 mtu 9000
d4 ifconfig d4-eth0 mtu 9000

s1 ifconfig s1-eth1 mtu 9000
s1 ifconfig s1-eth2 mtu 9000
s1 ifconfig s1-eth3 mtu 9000
s2 ifconfig s2-eth1 mtu 9000
s2 ifconfig s2-eth2 mtu 9000
s2 ifconfig s2-eth3 mtu 9000

r1 ifconfig r1-eth1 mtu 9000
r1 ifconfig r1-eth2 mtu 9000
r2 ifconfig r2-eth1 mtu 9000
r2 ifconfig r2-eth2 mtu 9000
```

Figura 35. Secuencia de comandos para configurar el MTU

Es posible comprobar que la red acepta un MTU más alto de lo previamente configurado enviando un *ping* con la opción de no fragmentar y un tamaño mayor como el de la *Figura 36*, dentro del nuevo límite establecido.

```
# Send big packet to test new MTU limitations
d1 ping -c 3 d4 -s 8000 -M do
```

Figura 36. Comando ping de tamaño incrementado sin fragmentación

Con la red preparada, en el diagrama mostrado es posible abrir terminales para cada equipo empleando el menú de herramientas mostrado al hacer clic derecho sobre un nodo, esto permitirá lanzar el software desde los equipos implicados.

Una vez ejecutado *Wireshark* y en modo de captura, se lanzan el servidor y cliente desde los equipos *d1* y *d4*, respectivamente.

Para la primera ejecución se ejecutó el servidor con la corrección de paquetes deshabilitada, al igual que la simulación de pérdidas, dado que la red configurada ya emula las pérdidas, aunque también es una alternativa viable para la realización de esta prueba.

Una vez transmitido el vídeo completo, observamos en la ventana de estadísticas de la *Figura 37* que efectivamente, la pérdida de paquetes observada es del 9%, una cifra que concuerda con el 10% configurado en la red. Al no tener habilitada la corrección de errores hacia adelante, ninguno de los paquetes de vídeo ha sido recuperado, obteniendo una pérdida real de 181 paquetes de 1883 transmitidos.

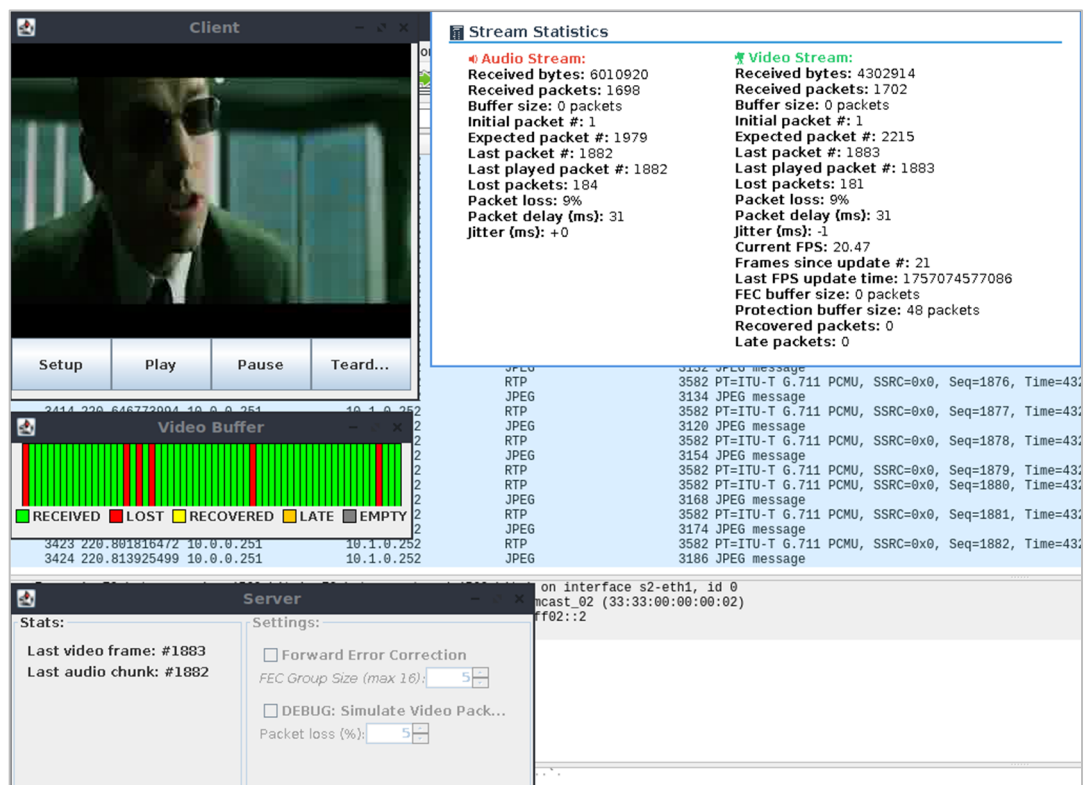
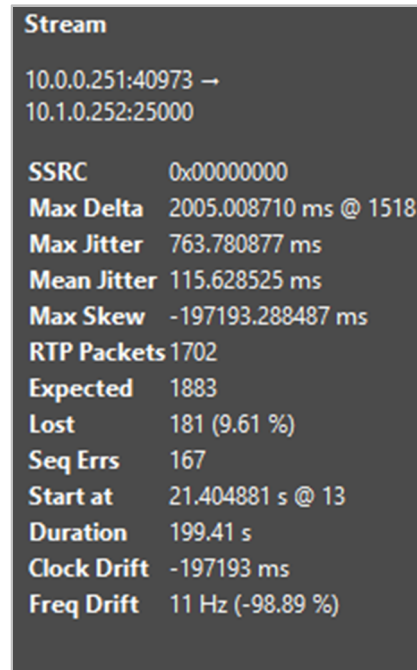


Figura 37. Estadísticas de transmisión sin FEC con 10% de pérdidas

Aunque no se está poniendo a prueba la transmisión de audio, podemos ver también una pérdida del 9%, lo que implica que las pérdidas afectan a ambos flujos, algo que resulta más relevante en las simulaciones con paquetes de redundancia.

En conjunto, un casi 10% de pérdidas deriva, tanto en vídeo como en audio, en una transmisión con interrupciones y saltos notables para el usuario, degradando la calidad de la experiencia con un flujo visiblemente errático.

Podemos comparar estas estadísticas obtenidas con las extraídas del flujo *RTP* de vídeo en la captura de *Wireshark*, el cual ha sido ejecutado sobre el *switch s2*. En la captura de pantalla de la *Figura 38* que la pérdida de paquetes concuerda con las de la aplicación.



Stream	
10.0.0.251:40973 → 10.1.0.252:25000	
SSRC	0x00000000
Max Delta	2005.008710 ms @ 1518
Max Jitter	763.780877 ms
Mean Jitter	115.628525 ms
Max Skew	-197193.288487 ms
RTP Packets	1702
Expected	1883
Lost	181 (9.61 %)
Seq Errs	167
Start at	21.404881 s @ 13
Duration	199.41 s
Clock Drift	-197193 ms
Freq Drift	11 Hz (-98.89 %)

Figura 38. Estadísticas de Wireshark en pérdidas sin FEC

Aunque inicialmente uno podría pensar que el *jitter* no coincide con las mostradas en la *Figura 37*, hay que tener en cuenta que la aplicación solo muestra el jitter del último paquete respecto al anterior paquete, no la media global. Además de esto, *Wireshark* también tiene en cuenta los paquetes perdidos a la hora de hacer el cálculo del *jitter*, lo que conlleva a una mayor media global en un escenario con pérdidas.

Tras la primera simulación, con el objetivo de comprobar la efectividad del *Forward Error Correction* a la hora de reducir el número de pérdidas, se realizó una segunda ejecución, bajo el mismo escenario, con *FEC* habilitado y un tamaño de grupo de cinco paquetes. A continuación, la *Figura 39* se muestran las estadísticas extraídas de la aplicación.

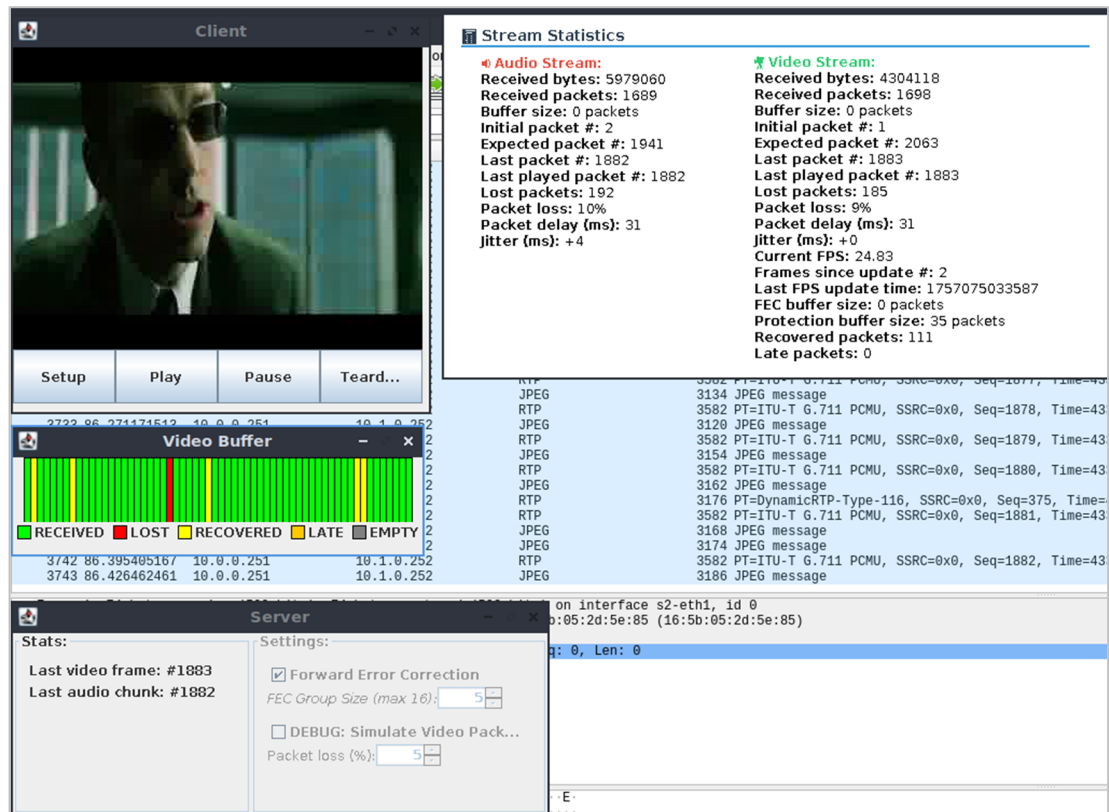


Figura 39. Estadísticas de transmisión con FEC de 5 paquetes con 10% de pérdidas

Dado que no se emplearon parámetros de lanzamiento, el servidor se lanzó con la corrección de errores habilitada y un tamaño de grupo predeterminado de cinco paquetes, aunque siempre es posible modificar estos valores desde la interfaz.

Realizada la transmisión completa del vídeo, se observan no solo algunos paquetes recuperados en el gráfico *Video Buffer* de la *Figura 39*, sino que se puede ver un total de 111 paquetes recuperados a pesar del 9% de pérdidas. Estas cifras suponen que hasta un 60% de los paquetes perdidos fueron recuperados exitosamente mediante la corrección de errores hacia adelante.

Por otra parte, observando detenidamente el gráfico de *Video Buffer* en la *Figura 39*, se puede ver un único paquete perdido, lo cual parece contraproducente teniendo en cuenta que en un radio de cuatro paquetes no hay otros paquetes perdidos, por lo que debería poder recuperarse, dado que los demás paquetes del grupo llegaron exitosamente. Sin embargo, hay que tener en cuenta que, como se mencionó durante el análisis de la *Figura 37*, las pérdidas de la red afectan a todos los flujos, esto incluye al flujo de paquetes de recuperación. Si analizamos las estadísticas extraídas por *Wireshark* sobre el flujo de paquetes de recuperación en la *Figura 40*, se ve claramente una pérdida cercana al 10%, además de que podemos identificar fácilmente el último paquete de recuperación perdido en la *Figura 41*, causante de que el fotograma no pudiera ser recuperado.

Stream	
10.0.0.251:41038 → 10.1.0.252:25000	
SSRC	0x00000000
Max Delta	1154.699256 ms @ 1505
Max Jitter	0.000000 ms
Mean Jitter	0.000000 ms
Max Skew	0.000000 ms
RTP Packets	341
Expected	376
Lost	35 (9.31 %)
Seq Errs	34
Start at	14.719065 s @ 22
Duration	71.61 s
Clock Drift	0 ms
Freq Drift	0 Hz (0.00 %)

Figura 40. Estadísticas de Wireshark con pérdidas y FEC de 5 paquetes en el flujo de redundancia

Paquete	Sequence	Delta (ms)	Jitter (ms)	Skew	Ancho de banda	Marker	Estado
3546	355	239.067526	0.000000	0.000000	67.39		✓
3563	357	499.293861	0.000000	0.000000	67.50		Wrong sequence number
3573	358	166.192749	0.000000	0.000000	90.10		✓
3584	359	166.428651	0.000000	0.000000	90.08		✓
3593	360	177.834558	0.000000	0.000000	90.53		✓
3602	361	153.503734	0.000000	0.000000	112.16		✓
3612	362	157.553492	0.000000	0.000000	133.76		✓
3632	364	310.520562	0.000000	0.000000	132.51		Wrong sequence number
3641	365	161.945148	0.000000	0.000000	131.20		✓
3650	366	207.227251	0.000000	0.000000	130.27		✓
3661	367	163.355355	0.000000	0.000000	127.02		✓
3671	368	156.210661	0.000000	0.000000	168.74		✓
3689	370	314.607238	0.000000	0.000000	167.86		Wrong sequence number
3700	371	780.576323	0.000000	0.000000	84.16		✓
3711	372	156.590545	0.000000	0.000000	126.37		✓
3721	373	169.737225	0.000000	0.000000	126.48		✓
3728	374	163.830014	0.000000	0.000000	168.62		✓
3738	375	162.225082	0.000000	0.000000	193.92		✓

Figura 41. Últimos paquetes de redundancia perdidos en pérdidas con FEC de 5 paquetes

Por supuesto, analizando el flujo *RTP* de vídeo en *Wireshark*, podemos extraer fácilmente el fotograma concreto perdido buscando por número de paquetes cercanos al del paquete de recuperación perdido. En la siguiente *Figura 42* se muestra que el fotograma con número de secuencia 1852 se perdió en el camino.

Paquete	Sequence	Delta (ms)	Jitter (ms)	Skew	Ancho de banda	Marker	Estado
3676	1848	31.082132	37.224596	-69311.335949	765.33		✓
3678	1849	34.708953	37.043063	-69345.656013	785.76		✓
3680	1850	31.812930	36.694652	-69377.124499	806.40		✓
3681	1851	30.618141	36.293342	-69407.398195	827.02		✓
3684	1853	62.824183	37.907770	-69469.522378	826.51		Wrong sequence number
3686	1854	30.969587	37.452606	-69500.147521	847.23		✓
3688	1855	30.482125	36.996117	-69530.296313	889.12		✓
3690	1856	419.885574	60.635042	-69945.515220	719.18		✓
3693	1857	36.955236	59.129359	-69982.059345	739.66		✓
3694	1858	246.388683	70.662234	-70225.714694	459.82		✓

Figura 42. Paquete de video perdido con FEC de 5 paquetes

Como ejemplo contrario, llegando al final de la transmisión encontramos dos paquetes perdidos que fue recuperado exitosamente, pues es mostrado en la *Figura 43*, correspondiente a la captura del flujo de vídeo en *Wireshark*, cómo los paquetes 1875 y 1876 se perdieron, sin embargo, tal y como se muestra en el gráfico de la *Figura 39*, los paquetes fueron recuperados exitosamente.

Paquete	Sequence	Delta (ms)	Jitter (ms)	Skew	Ancho de banda	Marker	Estado
3708	1864	31.216680	58.646237	-70426.619821	503.10		✓
3710	1865	30.759301	56.882470	-70457.045788	545.09		✓
3712	1866	37.726712	55.658846	-70494.350278	503.34		✓
3714	1867	38.061588	54.532629	-70531.989644	503.42		✓
3716	1868	31.504776	53.071166	-70563.138864	545.44		✓
3719	1869	31.067012	51.674378	-70593.861432	587.47		✓
3720	1870	31.391928	50.385197	-70624.908916	629.47		✓
3723	1871	32.071721	49.218383	-70656.625081	671.50		✓
3725	1872	30.271462	48.012673	-70686.552099	713.50		✓
3726	1873	38.755692	47.407722	-70724.885568	755.50		✓
3727	1874	32.144275	46.431535	-70756.674288	797.26		✓
3731	1877	93.664599	49.318323	-70849.294442	822.22		Wrong sequence number
3733	1878	37.211335	48.535942	-70886.094666	847.07		✓
3735	1879	30.873857	47.410534	-70916.624079	872.19		✓
3737	1880	31.082123	46.368481	-70947.361757	855.44		✓
3740	1881	31.419028	45.411918	-70978.425230	838.66		✓
3741	1882	30.670810	44.469765	-71008.762706	863.94		✓
3743	1883	31.245130	43.621003	-71039.652281	889.31		✓

Figura 43. Paquetes de video perdidos recuperados con FEC de 5 paquetes

En este caso, aunque tienen números de secuencia consecutivos, como pertenecen a dos grupos diferentes fueron exitosamente recuperados mediante *FEC*. Podemos identificar los paquetes de redundancia correspondientes en el flujo de paquetes *FEC* fácilmente empleando ambos flujos como filtro en *Wireshark*. Filtrando la captura con únicamente los paquetes del flujo de vídeo y redundancia, podemos observar los dos últimos paquetes *FEC* y vídeo en la *Figura 44* donde, si analizamos los dos últimos paquetes de redundancia de número de paquete 3728 y 3738, podemos comprobar el número de secuencia base de ambos en la carga *RTP*.

No.	Time	Source	Destination	Protocol	Sequence number	Length	Info
3727	86.140295579	10.0.0.251	10.1.0.252	JPEG	1874	5234	JPEG message
3728	86.170944181	10.0.0.251	10.1.0.252	RTP	374	5282	PT=DynamicRTP-Type-116, SSRC=0x0, Seq=374, Time=433392142
3731	86.233960178	10.0.0.251	10.1.0.252	JPEG	1877	3134	JPEG message
3733	86.271171513	10.0.0.251	10.1.0.252	JPEG	1878	3120	JPEG message
3735	86.302045370	10.0.0.251	10.1.0.252	JPEG	1879	3154	JPEG message
3737	86.333127493	10.0.0.251	10.1.0.252	JPEG	1880	3162	JPEG message
3738	86.333169263	10.0.0.251	10.1.0.252	RTP	375	3176	PT=DynamicRTP-Type-116, SSRC=0x0, Seq=375, Time=433392304
3740	86.364546521	10.0.0.251	10.1.0.252	JPEG	1881	3168	JPEG message
3741	86.395217331	10.0.0.251	10.1.0.252	JPEG	1882	3174	JPEG message
3743	86.426462461	10.0.0.251	10.1.0.252	JPEG	1883	3186	JPEG message

Figura 44. Extracto de últimos paquetes de vídeo y redundancia con FEC de 5 paquetes

Mostrando la información de la carga en bits, marcado en rojo, podemos encontrar el número de secuencia base de los paquetes y, como se muestra en la Figura 45 y la Figura 46, los números de secuencia en binario señalados en azul “00000111 01001111” y “00000111 01010100”, corresponden con los valores decimales de 1871 y 1876, respectivamente. Por lo que, los paquetes perdidos 1875 y 1876 identificados en la Figura 43 se encuentran en dos grupos distintos y, por tanto, fueron recuperados exitosamente gracias a *Forward Error Correction*.

0030	00001010	00001110	00000000	00000000	00000000	00000000	00000000	00011010
0038	00000111	01001111	00011001	11010101	00001010	00001101	00001111	01100100
0040	00010100	01011110	11111000	00000000	11111111	11011000	11111111	11100000
0048	00000000	00010000	01001010	01000110	01001001	01000110	00000000	00000001
0050	00000010	00000000	00000000	11100000	00000000	10101011	00000000	00000000

Figura 45. Extracto de carga de paquete de redundancia 3728

0030	00001010	10110000	00000000	00000000	00000000	00000000	00000000	00011010
0038	00000111	01010100	00011001	11010101	00001010	00110011	00001011	11001100
0040	00001100	00100100	11111000	00000000	11111111	11011000	11111111	11100000
0048	00000000	00010000	01001010	01000110	01001001	01000110	00000000	00000001
0050	00000010	00000000	00000000	11100000	00000000	10101011	00000000	00000000

Figura 46. Extracto de carga de paquete de redundancia 3738

Finalmente, para comprobar el efecto de modificar el tamaño del grupo *FEC*, se realizó, bajo el mismo escenario con pérdidas del 10%, otra simulación con un tamaño de grupo de tres paquetes, resultando en los resultados de la Figura 47. Los resultados muestran un incremento en la cantidad de paquetes perdidos, resultando en 151 paquetes recuperados de los 210 paquetes perdidos, es decir, empleando un grupo de paquetes menor se recuperó casi un 72%, una mejora a considerar respecto al porcentaje de 60% de la simulación con grupos de cinco paquetes.

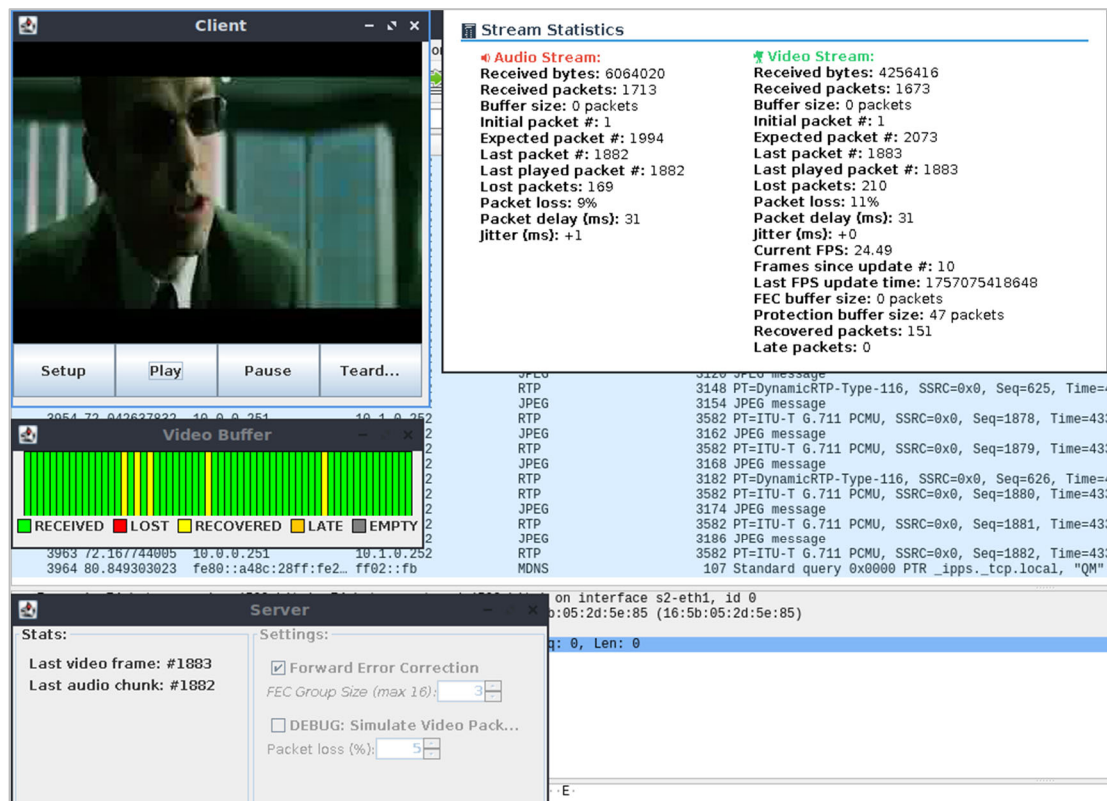


Figura 47. Estadísticas de transmisión con FEC de 3 paquetes con 10% de pérdidas

Aunque la recuperación de paquetes no consigue eliminar por completo en las simulaciones la pérdida de información, la mayoría de las interrupciones fueron subsanadas, mejorando la fluidez del vídeo y, por tanto, mejorando la calidad de la experiencia del usuario final.

Comparando los resultados de las pruebas mediante la *Tabla 5*, aunque destaca una mejora notable en el porcentaje de paquetes de vídeo recuperados, el uso de un grupo *FEC* más reducido implica un notable aumento, de casi el doble, en el número de paquetes de redundancia, lo que puede resultar inviable en situaciones donde la red sea limitada. Por otra parte, un grupo demasiado grande podría suponer una mayor cantidad de pérdidas irrecuperables, además de necesitar un mayor tiempo de espera al inicio para que el *buffer* pueda esperar a la llegada del paquete de redundancia, el cual es enviado justo al final del grupo.

	Sin FEC	FEC (3 paquetes)	FEC (5 paquetes)
Paquetes enviados	1883	1883	1883
Paquetes perdidos	181	210	185
Paquetes perdidos (%)	9,6%	11,2%	9,8%
Paquetes recuperados	0	151	111
Paquetes recuperados sobre perdidos (%)	0%	71,9%	60%
Paquetes de redundancia requeridos	0	628	377

Tabla 5. Resultados de las pruebas en el escenario 1

7.3 Escenario 2: Reducción de la variación del retardo mediante un *de-jitter buffer*

Con el objetivo de comprobar el funcionamiento del *de-jitter buffer*, bajo la misma topología, se agregó un *jitter* simulado de 60ms, una cifra bastante elevada con la que provocar bastantes paquetes fuera de orden. La inserción de *jitter* se realizó mediante la propiedad *delay* en el fichero de la topología, en la que se usó un retardo de 10ms, además del *jitter*. Como se muestra en el extracto de la topología en la Figura 48.

```
...
# Add router-router link in a new subnet for the router-router connection
self.addLink(r1,
              r2,
              intfName1='r1-eth2',
              intfName2='r2-eth2',
              params1={'ip': '10.100.0.1/24'},
              params2={'ip': '10.100.0.2/24'},
              delay='10ms 60ms')
...
```

Figura 48. Configuración de jitter en la topología emulada

Una vez configurado el *jitter* en el enlace, se lanza la topología mediante el comando de la *Figura 33*, se enruta las subredes mediante la secuencia de la *Figura 34* y finalmente, se incrementa el límite de *MTU* para evitar la fragmentación de los paquetes transmitidos usando los comandos de la *Figura 35*, tal y como se realizó en el anterior escenario.

Con el escenario preparado, se inicia el servidor en el host *d1* y el cliente en *d4*. Para esta prueba se deshabilitó *Forward Error Correction* para evitar su intervención en los paquetes temporalmente marcados como perdidos.

Desde el inicio de la transmisión empiezan a pronunciarse rápidamente los efectos de la alta variación de retardo, resultando en cientos de paquetes fuera de orden. Este síntoma se hacía notorio en el gráfico de *Video Buffer*, donde los paquetes desordenados se marcaban como perdidos durante un breve instante, para inmediatamente ser actualizados como recibidos, como se muestra en la *Figura 49* y la *Figura 50*.

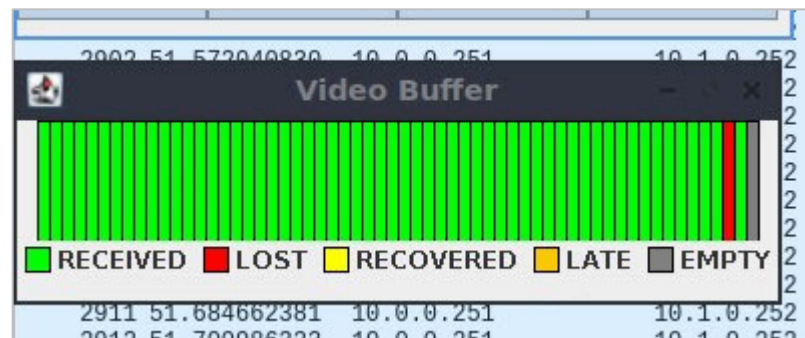


Figura 49. Fotograma temporalmente marcado como perdido

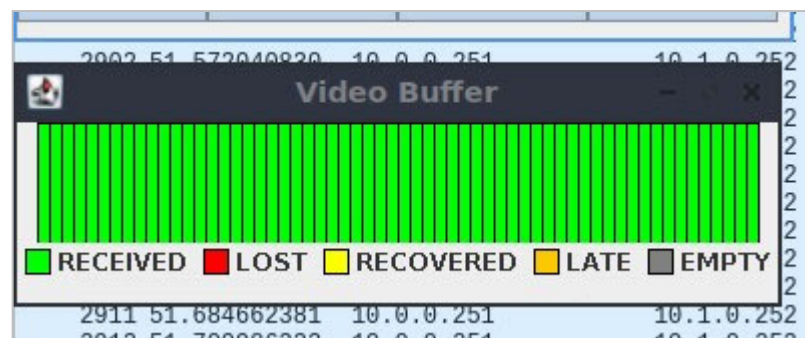


Figura 50. Estado de fotograma actualizado

Tras la transmisión completa, en total se recibieron 735 paquetes fuera de orden, los cuales fueron exitosamente reordenados antes de su reproducción gracias al *de-jitter buffer*. Todos los paquetes recibidos fuera de orden fueron mostrados en la consola habilitando el parámetro “-v”, como se muestra en el extracto de la *Figura 51*.

```
...
[UpdateStats] Lost 1 packets between 259 and 261
[UpdateStats] Out of order packet received: 260 (last: 261)
[UpdateStats] Lost 1 packets between 261 and 263
[UpdateStats] Out of order packet received: 262 (last: 263)
[UpdateStats] Lost 1 packets between 260 and 262
[UpdateStats] Out of order packet received: 261 (last: 262)
[UpdateStats] Lost 1 packets between 263 and 265
[UpdateStats] Lost 1 packets between 267 and 269
[UpdateStats] Out of order packet received: 264 (last: 265)
[UpdateStats] Out of order packet received: 268 (last: 269)
[UpdateStats] Lost 1 packets between 266 and 268
[UpdateStats] Lost 1 packets between 269 and 271
[UpdateStats] Out of order packet received: 267 (last: 268)
[UpdateStats] Out of order packet received: 270 (last: 271)
[UpdateStats] Lost 1 packets between 271 and 273
[UpdateStats] Out of order packet received: 272 (last: 273)
[UpdateStats] Lost 1 packets between 270 and 272
[UpdateStats] Out of order packet received: 271 (last: 272)
[UpdateStats] Lost 1 packets between 274 and 276
[UpdateStats] Out of order packet received: 275 (last: 276)
[UpdateStats] Lost 1 packets between 277 and 279
[UpdateStats] Out of order packet received: 278 (last: 279)
[UpdateStats] Lost 1 packets between 276 and 278
[UpdateStats] Out of order packet received: 277 (last: 278)
[UpdateStats] Lost 2 packets between 281 and 284
[UpdateStats] Out of order packet received: 282 (last: 284)
[UpdateStats] Out of order packet received: 283 (last: 284)
[UpdateStats] Lost 1 packets between 280 and 282
[UpdateStats] Out of order packet received: 281 (last: 282)
[UpdateStats] Lost 1 packets between 283 and 285
[UpdateStats] Out of order packet received: 284 (last: 285)
[UpdateStats] Lost 1 packets between 285 and 287
[UpdateStats] Out of order packet received: 286 (last: 287)
[UpdateStats] Lost 1 packets between 288 and 290
[UpdateStats] Out of order packet received: 289 (last: 290)
[UpdateStats] Lost 1 packets between 292 and 294
[UpdateStats] Lost 1 packets between 290 and 292
[UpdateStats] Out of order packet received: 293 (last: 295)
...
```

Figura 51. Extracto de consola de simulación con jitter

Adicionalmente, podemos comprobar que algunos paquetes se encuentran fuera de orden en las trazas capturadas mediante *Wireshark*, mostradas en la Figura 52, en la que se muestran algunos de los mismos paquetes reportados como fuera de orden marcados.

No.	Time	Source	Destination	Protocol	Sequence number	Length	Info
526	13.303808537	10.0.0.251	10.1.0.252	JPEG	258	1598	JPEG message
527	13.331158120	10.0.0.251	10.1.0.252	JPEG	259	1598	JPEG message
531	13.400967262	10.0.0.251	10.1.0.252	JPEG	261	1526	JPEG message
533	13.426838180	10.0.0.251	10.1.0.252	JPEG	260	1566	JPEG message
535	13.455290657	10.0.0.251	10.1.0.252	JPEG	263	1462	JPEG message
536	13.486151714	10.0.0.251	10.1.0.252	JPEG	262	1492	JPEG message
539	13.548237109	10.0.0.251	10.1.0.252	JPEG	264	1438	JPEG message
540	13.550925568	10.0.0.251	10.1.0.252	JPEG	265	1424	JPEG message
541	13.573876004	10.0.0.251	10.1.0.252	JPEG	266	1392	JPEG message
543	13.582021945	10.0.0.251	10.1.0.252	JPEG	267	1358	JPEG message
545	13.643609007	10.0.0.251	10.1.0.252	JPEG	269	1304	JPEG message
547	13.664804187	10.0.0.251	10.1.0.252	JPEG	268	1336	JPEG message
550	13.706019743	10.0.0.251	10.1.0.252	JPEG	271	1234	JPEG message
552	13.709618207	10.0.0.251	10.1.0.252	JPEG	270	1276	JPEG message
554	13.771371706	10.0.0.251	10.1.0.252	JPEG	273	1178	JPEG message
556	13.780505623	10.0.0.251	10.1.0.252	JPEG	272	1212	JPEG message
557	13.801870811	10.0.0.251	10.1.0.252	JPEG	274	1156	JPEG message
560	13.863708106	10.0.0.251	10.1.0.252	JPEG	276	1110	JPEG message
561	13.872693100	10.0.0.251	10.1.0.252	JPEG	275	1138	JPEG message
564	13.910136959	10.0.0.251	10.1.0.252	JPEG	277	1078	JPEG message
566	13.971209311	10.0.0.251	10.1.0.252	JPEG	279	1024	JPEG message
568	13.988544767	10.0.0.251	10.1.0.252	JPEG	278	1048	JPEG message
569	14.006140640	10.0.0.251	10.1.0.252	JPEG	280	1010	JPEG message
571	14.022917883	10.0.0.251	10.1.0.252	JPEG	281	994	JPEG message
575	14.118590866	10.0.0.251	10.1.0.252	JPEG	284	1194	JPEG message
576	14.120173242	10.0.0.251	10.1.0.252	JPEG	282	978	JPEG message
577	14.148360305	10.0.0.251	10.1.0.252	JPEG	283	996	JPEG message
580	14.169748276	10.0.0.251	10.1.0.252	JPEG	285	1252	JPEG message
582	14.193308755	10.0.0.251	10.1.0.252	JPEG	286	1364	JPEG message
583	14.218121157	10.0.0.251	10.1.0.252	JPEG	287	1424	JPEG message
584	14.241362305	10.0.0.251	10.1.0.252	JPEG	288	1496	JPEG message
587	14.272736104	10.0.0.251	10.1.0.252	JPEG	289	1628	JPEG message
589	14.368643443	10.0.0.251	10.1.0.252	JPEG	290	1738	JPEG message
592	14.382381103	10.0.0.251	10.1.0.252	JPEG	291	1816	JPEG message
593	14.401284365	10.0.0.251	10.1.0.252	JPEG	292	1856	JPEG message
596	14.495369903	10.0.0.251	10.1.0.252	JPEG	294	2066	JPEG message
598	14.495764746	10.0.0.251	10.1.0.252	JPEG	295	2096	JPEG message
599	14.498695185	10.0.0.251	10.1.0.252	JPEG	293	2016	JPEG message

Figura 52. Paquetes fuera de orden capturados en *Wireshark*

Cabe añadir, que algunos paquetes podrían haber sido recibidos en distinto orden por la aplicación frente a *Wireshark*, esto puede suceder debido a que se realiza la captura de paquetes sobre el *switch s2* y por la forma en la que *Java* gestiona los paquetes que llegan prácticamente de forma simultánea.

De cualquier forma, incluso en un entorno de alta variación de *jitter* como el planteado, el *de-jitter buffer* ha sido capaz de corregir todos los problemas de paquetes fuera de orden y la experiencia de reproducción de la transmisión resultó completamente fluida, sin saltos ni pérdidas en la sincronización. El uso del *buffer* demuestra ser crítico a la hora de asegurar una reproducción fluida y se muestra como un elemento decisivo a la hora de determinar la calidad de experiencia final del usuario.

8. CONCLUSIONES

A lo largo del presente trabajo se ha logrado cumplir con los objetivos propuestos, desarrollando e integrando en un proyecto técnicas destinadas a la mejora de calidad de transmisión de flujos multimedia en tiempo real. Partiendo de un proyecto base limitado, basado en los protocolos estándar *RTP* y *RTSP*, se implementó una arquitectura cliente-servidor que, no solo soporta de forma simultánea el envío y recepción de audio y vídeo, sino que además incorpora mecanismos avanzados de corrección de errores y estabilización de la transmisión.

El desarrollo de este proyecto ha permitido adquirir un conocimiento profundo acerca de conceptos fundamentales de la transmisión multimedia, abordando desde términos como el retardo o métricas de *Quality of Service (QoS)* y *Quality of Experience (QoE)*, hasta técnicas de gestión de *jitter* y pérdidas de información en tiempo real. Todo este marco teórico ha sido también empleado en una aplicación práctica diseñada específicamente para evaluar dichos conceptos sobre escenarios controlados, empleando para ello herramientas conocidas como *Mininet* para la emulación de redes y *Wireshark* para la extracción detallada de información de la red.

La implementación del *de-jitter buffer*, aunque esencialmente simple, ha demostrado ser un componente esencial para compensar las variaciones de retardo producidas por la red, permitiendo una reproducción fluida y en sincronía aún en entornos inestables.

Por otra parte, la integración de la corrección de errores mediante *Forward Error Correction (FEC)*, basado sobre el estándar *RFC 5109*, ha permitido implantar un sistema de redundancia controlada que permite la recuperación de información relevante sin necesidad de retransmisiones. Mostrando prometedores resultados con altos porcentajes de recuperación en las pruebas realizadas sobre una red con pérdidas, presentando ser una solución viable para mejorar la robustez de la transmisión en tiempo real en situaciones donde la latencia es un factor crítico y la retransmisión no es posible.

El proyecto no se ha limitado únicamente al estudio e implementación de estas técnicas, sino que también ha supuesto una reestructuración de la arquitectura del proyecto original, optimizando el uso de hilos de ejecución, separando y sincronizando correctamente los flujos de audio y vídeo e incorporando mejoras significativas en la interfaz y estadísticas en tiempo real. Estas mejoras no solo facilitan el análisis del funcionamiento de las técnicas implementadas y la transmisión, sino que sientan las bases para futuros desarrollos más complejos, orientados a la optimización y mejora de transmisiones multimedia en tiempo real.

En definitiva, este trabajo no solo ha cumplido los objetivos iniciales, sino que ha concluido en una base sólida y escalable sobre la que pueden construirse futuras soluciones. La arquitectura desarrollada, junto con los mecanismos implementados, ofrecen un punto de partida idóneo para la exploración de estrategias más avanzadas de optimización de las transmisiones. De esta forma, aporta una perspectiva experimental al marco teórico ya existente y se establece como una base práctica para futuras investigaciones y desarrollos en el ámbito de las comunicaciones multimedia.

8.1 Trabajos futuros

Teniendo en cuenta la base estructural que proporciona este trabajo, se propone para futuros trabajos la implementación de técnicas avanzadas, como *buffering* dinámico, el uso de resolución adaptativa, el estudio de códigos de corrección más avanzados, como los mencionados *Reed-Solomon* o *LDPC*, además de la posibilidad de implementar una mejor gestión de la transmisión mediante *RTCP*.

Alternativamente sería posible indagar en los efectos sobre la *QoE* provocados por cambios en las métricas *QoS*, relacionando así las métricas objetivas con mediciones subjetivas como el *MOS*. Permitiendo así, arrojar luz sobre la relación entre los valores estadísticos y la calidad de la experiencia final del usuario.

9. REFERENCIAS BIBLIOGRÁFICAS

- [1] Network Startup Resource Center, «Network Delay,» Network Startup Resource Center, [En línea]. Available: <https://nsrc.org/workshops/2012/drukren-nsrc/raw-attachment/wiki/Agenda/types-of-delay.pdf>. [Último acceso: 1 Abril 2025].
- [2] P. J. B. Pérez, «Fundamentos de Redes,» [En línea]. Available: <https://pedrobq.gitbooks.io/fundamentos-de-redes/content/>. [Último acceso: 1 Abril 2025].
- [3] c. d. Wikipedia, «Calidad de servicio,» Wikipedia, La enciclopedia libre, 1 Septiembre 2024. [En línea]. Available: https://es.wikipedia.org/w/index.php?title=Calidad_de_servicio&oldid=162192611. [Último acceso: 31 Agosto 2025].
- [4] Linkedin, «Métricas y estándares de QoS de vídeo para proyectos de vídeo sobre IP,» Linkedin, [En línea]. Available: <https://www.linkedin.com/advice/3/what-key-video-qos-metrics-standards-you-use-follow>. [Último acceso: 31 Agosto 2025].
- [5] N. Agarwal, «Computer Network | Quality of Service and Multimedia,» GeeksforGeeks, 11 Julio 2025. [En línea]. Available: <https://www.geeksforgeeks.org/computer-networks/computer-network-quality-of-service-and-multimedia/>. [Último acceso: 31 Agosto 2025].
- [6] W. contributors, «RTP Control Protocol,» Wikipedia, The Free Encyclopedia, 13 Agosto 2025. [En línea]. Available: https://en.wikipedia.org/w/index.php?title=RTP_Control_Protocol&oldid=1305732686. [Último acceso: 31 Agosto 2025].
- [7] c. d. Wikipedia, «Calidad de experiencia,» Wikipedia, La enciclopedia libre, 13 Mayo 2024. [En línea]. Available: https://es.wikipedia.org/w/index.php?title=Calidad_de_experiencia&oldid=160086414. [Último acceso: 28 Agosto 2025].
- [8] Qualinet, «Qualinet White Paper on Definitions of Quality of Experience Output from the fifth Qualinet meeting, Novi Sad,» 12 Marzo 2013. [En línea]. Available: https://www.qualinet.eu/wp-content/uploads/2021/04/QoE_whitepaper_v1.2.pdf. [Último acceso: 28 Agosto 2025].
- [9] GeeksforGeeks, «Difference between Quality of Service (QoS) and Quality of Experience (QoE),» GeeksforGeeks, 23 Julio 2025. [En línea]. Available: <https://www.geeksforgeeks.org/computer-networks/difference-between->

- quality-of-service-qos-and-quality-of-experience-qoe/. [Último acceso: 30 Agosto 2025].
- [10] W. contributors, «Mean opinion score,» Wikipedia, The Free Encyclopedia, 25 Enero 2025. [En línea]. Available: https://en.wikipedia.org/w/index.php?title=Mean_opinion_score&oldid=1271816844. [Último acceso: 30 Agosto 2025].
- [11] C. A. I. C. A. C. Jesús Moya Neyra, «Evaluación de QoE en servicios IP basada en parámetros de QoS | Revista Ingeniería Electrónica, Automática y Comunicaciones ISSN: 1815-5928,» 15 Febrero 2019. [En línea]. Available: <https://rielac.cujae.edu.cu/index.php/rieac/article/view/481>. [Último acceso: 30 Agosto 2025].
- [12] N. Doroshni, «Understanding Delay in Packet Voice Networks,» Cisco, 2 2 2006. [En línea]. Available: <https://www.cisco.com/c/en/us/support/docs/voice/voice-quality/5125-delay-details.html>. [Último acceso: 11 04 2025].
- [13] c. d. Wikipedia, «Búfer de datos - Wikipedia, la enciclopedia libre,» Wikipedia, La enciclopedia libre, 18 Diciembre 2024. [En línea]. Available: https://es.wikipedia.org/wiki/Búfer_de_datos. [Último acceso: 27 Abril 2025].
- [14] Cloudflare, «¿Qué significa "almacenamiento en búfer"? | Almacenamiento en vídeo en la transmisión de vídeo | Cloudflare,» Cloudflare, [En línea]. Available: <https://www.cloudflare.com/es-es/learning/video/what-is-buffering/>. [Último acceso: 27 Abril 2025].
- [15] Conviva, «What Is Buffering? - Conviva,» [En línea]. Available: <https://www.conviva.com/glossary/what-is-buffering/>. [Último acceso: 27 Abril 2025].
- [16] Akamai, «¿Qué es un almacenamiento en búfer? | ¿Cómo funciona el almacenamiento en búfer? | Akamai,» Akamai, [En línea]. Available: <https://www.akamai.com/es/glossary/what-is-buffering>. [Último acceso: 27 Abril 2025].
- [17] c. d. Wikipedia, «Protocolo de transporte en tiempo real,» Wikipedia, La enciclopedia libre, 24 Marzo 2025. [En línea]. Available: https://es.wikipedia.org/w/index.php?title=Protocolo_de_transporte_en_tiempo_real&oldid=166362408. [Último acceso: 5 Mayo 2025].
- [18] W. contributors, «Real-time Transport Protocol,» Wikipedia, The Free Encyclopedia, 28 Marzo 2025. [En línea]. Available: https://en.wikipedia.org/w/index.php?title=Real-time_Transport_Protocol&oldid=1282745499. [Último acceso: 5 Mayo 2025].
- [19] c. d. Wikipedia, «Protocolo de transmisión en tiempo real,» Wikipedia, La enciclopedia libre, 11 Junio 2025. [En línea]. Available:

- https://es.wikipedia.org/w/index.php?title=Protocolo_de_transmisi%C3%B3n_en_tiempo_real&oldid=167911691. [Último acceso: 19 Agosto 2025].
- [20] S. C. R. F. V. J. H. Schulzrinne, «RFC 3550: RTP: A Transport Protocol for Real-Time Applications,» Julio 2003. [En línea]. Available: <https://www.rfc-editor.org/rfc/rfc3550>. [Último acceso: 18 Agosto 2025].
- [21] c. d. Wikipedia, «Protocolo de control en tiempo real,» Wikipedia, La enciclopedia libre, 5 Octubre 2022. [En línea]. Available: https://es.wikipedia.org/w/index.php?title=Protocolo_de_control_en_tiempo_real&oldid=146399556. [Último acceso: 7 Septiembre 2025].
- [22] K. L. y. E. T. Wahul Awati, «What is Real-Time Transport Control Protocol (RTCP)?,» TechTarget, 19 Agosto 2024. [En línea]. Available: <https://www.techtarget.com/searchunifiedcommunications/definition/Real-Time-Transport-Control-Protocol-RTCP>. [Último acceso: 7 Septiembre 2025].
- [23] S. H. C. S. F. R. y. V. J. Audio-Video Transport Working Group, «RFC 1889 - RTP: A Transport Protocol for Real-Time Applications,» Enero 1996. [En línea]. Available: <https://www.rfc-editor.org/rfc/rfc1889>. [Último acceso: 28 Julio 2025].
- [24] W. contributors, «Real-Time Streaming Protocol,» Wikipedia, The Free Encyclopedia, 23 Junio 2023. [En línea]. Available: https://en.wikipedia.org/w/index.php?title=Real-Time_Streaming_Protocol&oldid=1295418749. [Último acceso: 20 Agosto 2025].
- [25] H. R. A. y. R. L. Schulzrinne, «RFC 2326: Real Time Streaming Protocol (RTSP),» IETF, Abril 1998. [En línea]. Available: <https://www.rfc-editor.org/info/rfc2326>. [Último acceso: 20 Agosto 2025].
- [26] H. R. A. L. R. W. M. y. M. S. E. Schulzrinne, «RFC 7826: Real-Time Streaming Protocol Version 2.0,» IETF, Diciembre 2016. [En línea]. Available: <https://www.rfc-editor.org/info/rfc7826>. [Último acceso: 20 Agosto 2025].
- [27] c. d. Wikipedia, «Corrección de errores hacia adelante,» Wikipedia, La enciclopedia libre, 26 Noviembre 2024. [En línea]. Available: https://es.wikipedia.org/w/index.php?title=Correcci%C3%B3n_de_errores_hacia_adelante&oldid=163788908. [Último acceso: 30 Agosto 2025].
- [28] R. Sheldon, «What is forward error correction (FEC)? | Definition from TechTarget,» TechTarget, 22 Agosto 2023. [En línea]. Available: <https://www.techtarget.com/searchmobilecomputing/definition/forward-error-correction>. [Último acceso: 30 Agosto 2025].

- [29] Jomplair, «Corrección de errores hacia adelante (FEC),» Jomplair, 26 Febrero 2025. [En línea]. Available: <https://www.jomplair.com/blog/forward-error-correction-fec>. [Último acceso: 30 Agosto 2025].
- [30] E. A. Li, «RFC 5109: RTP Payload Format for Generic Forward Error Correction,» Diciembre 2007. [En línea]. Available: <https://www.rfc-editor.org/rfc/rfc5109>. [Último acceso: 2 Agosto 2025].
- [31] Terrazone, «Forward Error Correction: The Unsung Hero of WAN Optimization,» Terrazone, 14 Mayo 2025. [En línea]. Available: <https://terrazone.io/forward-error-correction/>. [Último acceso: 30 Agosto 2025].
- [32] Mininet, «Mininet: An Instant Virtual Network on your Laptop (or other PC),» Mininet, [En línea]. Available: <https://mininet.org/>. [Último acceso: 1 Septiembre 2025].
- [33] The Mobile & Internet Systems Laboratory (MISL), «MiniNAM: A Network Animator for Mininet,» The Mobile & Internet Systems Laboratory (MISL), 5 Noviembre 2017. [En línea]. Available: <https://github.com/uccmisl/MiniNAM>. [Último acceso: 1 Septiembre 2025].
- [34] J. J. Q. y. C. J. S. Ahmed Khalid, «MiniNAM: A network animator for visualizing real-time packet flows in Mininet,» IEEE, 17 Abril 2017. [En línea]. Available: <https://ieeexplore.ieee.org/document/7899417>. [Último acceso: 1 Septiembre 2025].
- [35] «Wireshark - Go Deep,» Wireshark Foundation, [En línea]. Available: <https://www.wireshark.org/>. [Último acceso: 1 Septiembre 2025].